从第一步讲起

数据结构

超级纲要 · 教材式初稿

概念 · 推导 · 原图 · 例题 · 自测

> 先弄懂问题，再看图，再一步一步算。
> 定义、公式、代码、例题和错误，都放在一起解释。

2026-10-25 · sTBook
Wiki 与 Typst 共用正文 · 原图均在图注中注明出处

章节目录

1 数据结构：先知道要解决什么，再决定把数据摆成什么样 3
2 算法不只要跑出答案，还要说清为什么会结束 4
3 复杂度：把循环次数真正加出来 5
4 动态规划：别把同一道小题重新算一百遍 7
5 向量：连续座位好找人，插队就得挪座位 9
6 二分查找：先说清“找哪个边界” 10
7 排序：除了排好，还要知道搬了多少次 11
8 列表：改链接很快，找到位置可能很慢 12
9 栈：最后放进去的，最先拿出来 13
10 队列与单调栈：什么时候才能放心丢掉一个候选 14
11 二叉树：每一棵子树都还是同一种问题 16
12 Huffman：让常见字符少走几步 18
13 搜索树：别让一条长链假装成一棵树 19
14 高级搜索结构：不同方式把路径缩短 21
15 搜索树应用：一个点能找到，一大片范围怎么找 22
16 散列：先把键分到格子里，撞车再处理 23
17 图基础：先记录访问过谁，再沿边走 24
18 图进阶：最短的路和最省的连线不是同一个目标 25
19 优先级队列：每次先取最重要的，不必把全部排好 26
20 字符串匹配：已经比较过的信息，别白白扔掉 27
21 排序与选取：只找第 k 小，不必把所有人排队 29
22 PA1 五个主题：把讲解里的省略补出来 30
23 PS01：算法、级数与递归，32 题逐项检查 32
24 PS02：向量与查找，43 题逐项检查 35
25 PS03：列表与排序，37 题逐项检查 39
26 PS04：栈队列与单调结构，35 题逐项检查 43
27 PS05：二叉树与 Huffman，37 题逐项检查 47
28 从算法到程序：把边界当成题目的一部分 51
29 几个不能只背结论的证明 52
30 综合练习：先自己走一遍，再对答案 53
31 PS04 的 21 个表达式：RPN 与检查过程 55
32 从样卷提炼通用方法：不靠记住原题数字 58
33 资料覆盖、原图出处与使用说明 59

1 数据结构：先知道要解决什么，再决定把数据摆成什么样

你要反复查第 100 个元素，和你要反复摘掉手边一个节点，适合的存储方式不同。数据结构研究的是这些取舍；算法研究的是如何利用它们完成任务。本课程名称保留“数据结构”，不擅自加成另一门课。

1.1 四条线一起读

正文按课程大纲 14 章安排，并与 2026 版教材、已发布分章课件、PS01–PS05、PA1 两份讲解交叉核对。每份习题单有独立逐题导航。导航不是把题号勾掉就算完成：它指出该题在考什么、先证明哪句话、用什么小例子检查。教材中未提供原题的外部《习题解析》编号不冒充已读取。

第一遍读算法概论到二叉树，把当前课程根基打牢；第二遍读搜索树、散列和图；最后读高级结构与串。PA 章节用独立小数据演示思路，不拿你的个人提交当教材。实验报告和私人答题记录仍保持本地隐私规则。

1.2 每次分析都填四格

- **输入是什么，范围多大？** n 是元素数、位数还是顶点数？一个 1000 位整数不等于一个 RAM 字。
- **输出到底要什么？** 要值、位置、全部答案还是方案数？并列答案如何选？
- **为什么不会算错？** 写出已经完成的部分、尚未完成的部分和每一步保持的条件。
- **付出多少成本？** 分开计算比较、移动、额外空间；说明最坏、期望、摊还属于哪一种。

1.3 把术语翻译成人话

ADT（抽象数据类型）先说“能做哪些操作及其效果”；具体实现再说“用数组还是节点来做”。秩是从头数第几个，位置通常是节点的句柄；它们可能指向同一个元素，但获得它们的成本不同。复杂度中的 O 不是运行秒数，也不是“差不多”；它表达规模增大后的上界。下面会一步一步把这些话算成具体例子。

2 算法不只要跑出答案，还要说清为什么会结束

2.1 一份能执行的说明书

“把锅烧热，放适量盐”适合人做饭，却缺少机器需要的精确定义。算法需要明确步骤、可行的基本操作、输入输出约定和终止保证。持续运行的服务器程序有价值，但其无限运行过程不必符合入门教材对“有限算法”的定义。

曹冲称象中水位把重量转成可比较的量，替换物体时仍保存相关条件；刻舟求剑却把移动后的船上刻线当成固定地面位置，破坏了对应关系。关键不在“画横线还是竖线”，而在状态变化后，标记是否仍代表同一个待求量。

2.2 用循环不变式证明，不靠“测了几次”

例：求前 n 个数的和。开始 s=0、i=0；每轮把 a[i] 加进 s，再把 i 加一。不变式是：每轮开头 $s = \sum_{j=0}^{i-1} a_j$。初始化时空和为 0；执行一轮后恰好多加入 a[i]，所以下一轮仍成立；结束时 i=n，故 s 是全部和。

但不变式只说明“如果结束，结果对”。要证明会结束，还要给出下降量，例如 n-i：它是非负整数，每轮严格减一，不能无限下降。这把部分正确性补成完全正确性。

2.3 模型不同，“一步”的含义就不同

RAM 模型把规定字宽内的基本访问和算术看作常数成本，方便分析数据结构。图灵机一次只读写一个带格并移动读写头，更接近位级计算模型。若整数长度随输入增长，乘两个大整数不能无条件算 O(1)。先说明字长足够保存下标和普通元素，再使用单位成本假设。

复杂度也依赖输入编码。枚举 1 到 N 的算法按数值是 O(N)，但输入 N 只需约 $\log_2 N$ 位，所以按输入位数看可能是指数时间。这就是不能把“看起来只有一层循环”直接叫高效的原因。

2.4 几何分布与停机问题

独立重复试验每次成功概率 p，首次成功所需次数 K 满足 $P(K = k) = (1 - p)^{k-1}p$，期望 $E[K] = 1/p$。概率为 1 终止，不意味着存在统一有限最坏步数；随机算法必须说明概率模型。

不存在一个能对任意程序及输入都正确判定是否停机的通用算法。直观矛盾构造是：假设有判定器，再造一个程序，判定它会停就故意不停，判定它不停就立即停；让程序检查自身，两个答案都会矛盾。有限测试能发现问题，不能彻底消除所有程序的无限循环。

自测。“对一百万个输入都终止”能否证明所有自然数输入都终止？答：不能。Hailstone/Collatz 就提醒我们，实验和一般证明不能混为一谈。

3 复杂度：把循环次数真正加出来

3.1 O、Ω、Θ 分别承诺什么

$f(n) = O(g(n))$ 表示足够大时 $f(n) \le Cg(n)$，C 是固定常数；Ω 给下界；Θ 是上下界同时成立。$n = O(n^2)$ 也对，只是不紧。小 o 表示比值趋于 0，小 ω 表示比值趋于无穷。说一个算法“O(n²)”不能自动推断它最好也要 n² 步。

常数只能独立于输入规模。`n/1000` 仍是 Θ(n)，`n/log n` 不能把分母当“一个常数”。两层循环也不总是 n²：必须看每轮各执行多少次。

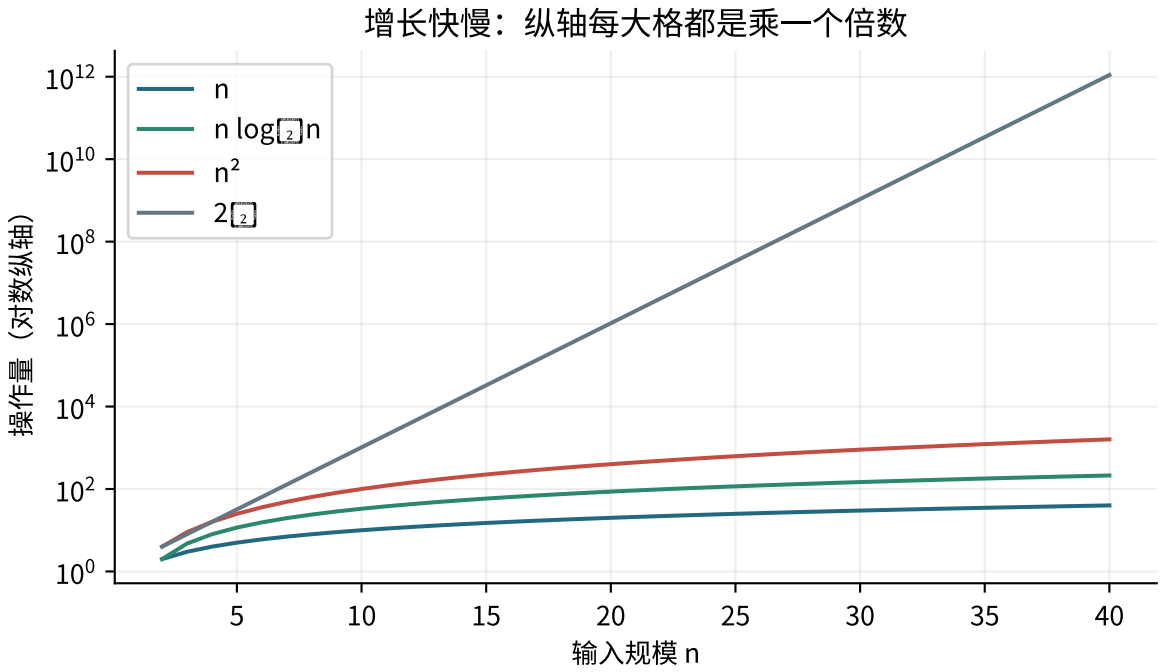


图 1　本书原创图：常见增长率，注意纵轴是对数刻度。

3.2 常见级数，给你把来历写出来

计数式	数量级	原因
$1 + 2 + \cdots + n$	Θ(n²)	首尾配对，每对约 n
$1 + 2 + 4 + \cdots + 2^k$	Θ(2^k)	和为 $2^{k+1} - 1$
$1 + 1/2 + \cdots + 1/n$	Θ(log n)	与 $\int_1^n dx/x$ 上下夹住
$\sum_{i=1}^{n} \log i$	Θ(n log n)	等于 log(n!)；后半项均至少 log(n/2)
$\sum_{i\ge1} 1/i^2$ 的前 n 项	Θ(1)	积分尾部收敛

分数可以估算整数操作次数：$\sum \lfloor n/i \rfloor$ 夹在 $nH_n - n$ 与 nH_n 之间，所以是 Θ(n log n)。不是说机器做了半次操作，而是我们用连续或分数界来估总数。

例：外层 i 从 1 到 n，内层 j 从 i 到 n、步长 i，执行约 n/i 次。总量是 $\sum_i \lfloor n/i \rfloor$，不是 n²。

3.3 递归树怎么数

归并型递推 $T(n) = 2T(n/2) + cn$：每层总工作 cn，共约 log₂n 层，叶层 O(n)，所以 Θ(n log n)。若总是分成 n/4 与 3n/4，每层总规模仍 n，深度介于两个常数倍 log n 之间，依然 Θ(n log n)。若分成 1 与 n-1，就可能退化为 Θ(n²)。

主定理常用形式 $T(n)=aT(n/b)+f(n)$，比较 $f(n)$ 与 $n^{\log_b a}$。当 $f(n)=\Theta(n^d)$ 时：d 小于临界值，叶子主导；等于时多乘 log n；大于时顶层主导，得到 Θ(n^d)。一般形式的“顶层主导”还需正则条件 $af(n/b)\le cf(n)$，c<1，不能随意省略。

Akra–Bazzi 处理不等分递推：在满足适用的增长与平滑条件、$\sum a_i b_i^p=1$ 时，$T(x)=\Theta\left(x^p(1+\int_1^x g(u)/u^{p+1}\,du)\right)$。它不是对所有递推都能硬套的口诀。对 $T(n)=T(n/4)+T(3n/4)+n$，p=1，积分为 log n，再次得 Θ(n log n)。

自测。 `for(i=1;i<n;i*=2)` 嵌套一个执行 i 次的循环，总量多少？答：$1+2+4+\cdots<n$，为 Θ(n)，不是 Θ(n log n)。实际代码还需防 i 翻倍溢出。

4 动态规划：别把同一道小题重新算一百遍

4.1 递归深度和调用总数分开看

朴素 Fibonacci 定义 $F_n = F_{n-1} + F_{n-2}$，会反复计算同一个 F_k。调用树很大，但同时活着的一条路径只有 O(n) 层。因此时间指数级，栈空间 O(n)。记忆化把每个 n 的答案存起来，状态 O(n)、每状态 O(1) 转移，时间和缓存空间均 O(n)。迭代只保留前两项可把辅助空间降为 O(1)，但大整数位成本另算。

动态规划必须说清状态、转移、边界、计算顺序。它不是“用了一个数组”就自动正确；如果状态缺少影响未来的信息，即使转移写得漂亮也会错。

4.2 最大和区间，四种想法是怎样省下来的

朴素枚举左右端点再重算区间和，$O(n^3)$；固定左端向右累加，$O(n^2)$；分治把最优区间分为左边、右边、跨中点，O(n log n)；线性扫描只记录“必须以当前位置结尾”的最优和。

允许空区间时，令 $e_i = \max(0, e_{i-1} + a_i)$，全局答案是所有 e_i 的最大值。若不允许空区间，改为 $e_i = \max(a_i, e_{i-1} + a_i)$，初值为首元素，不能从 0 开始，否则全负数组会错误返回空区间。

例：[−2,3,-1,4,-5]。非空版本的 e 依次为 −2、3、2、6、1，最优 6，对应 [3,-1,4]。要返回位置，就在选择“从当前位置重新开始”时记录左端，在全局更新时保存左右端。并列时应明定“和最大、左端最小、再右端最小”等顺序，而不是依赖偶然的 > 或 >=。

4.3 LCS：子序列不是连续子串

X=ABC，Y=AC，LCS 长度是 2，对应 A、C，不要求在 X 中连续。设 D[i][j] 表示前 i、前 j 个字符的 LCS 长度；空前缀为 0。末字符相同，取 $D[i-1][j-1]+1$；否则取 $\max(D[i-1][j], D[i][j-1])$。

填表时按 i、j 递增，依赖都已算好；时间 O(mn)，存整表 O(mn)，只要长度可滚动两行到 O(min(m,n))。要恢复序列，则从右下沿转移往回走；滚动数组会丢路径，不能只说“空间优化后仍直接回溯”。可保留方向或用 Hirschberg 类分治重构，但需要额外解释与实现。

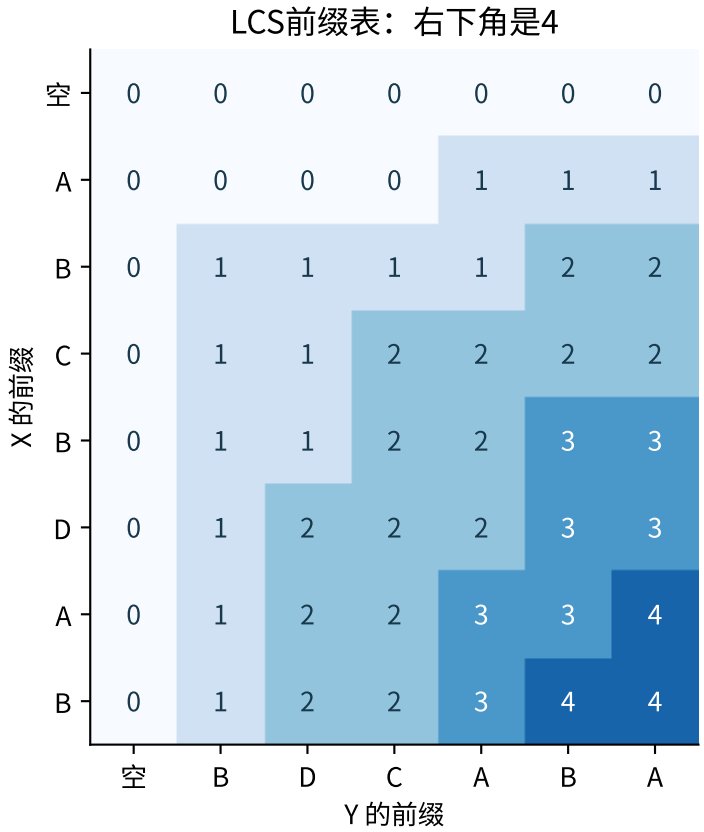


图 2　本书原创图：逐格计算 LCS，颜色只是辅助，格中数值才是状态。

表的空行/空列全 0；每格只依赖左、上或左上，右下角 4 是这两串的 LCS 长度。恢复具体序列时需要沿转移回走，不能只读这一格猜字符。

自测。 记忆化与自底向上会不会产生不同数学答案？答：若状态与递推完全一致，应一致；区别在访问顺序、是否跳过无关状态、栈成本与局部性。

5 向量：连续座位好找人，插队就得挪座位

5.1 size 与 capacity 不一样

size 是已有元素数，capacity 是当前存储能容纳的元素数。按秩访问 a[r] 可通过起点加偏移 O(1) 完成。若在 r 处插入 k 个元素，需要把后缀挪开，再写入 k 个元素，代价 O(n-r+k)。从后往前搬后缀，避免覆盖尚未搬走的元素；有重叠内存不能直接按普通 memcpy 的前提处理。

批量插入一次搬后缀即可；若循环调用单元素插入，可能反复搬同一大段，代价升为 Θ(k(n-r)+k²)。接口先实现批量版本，再包装单元素版本，有实际效率意义。

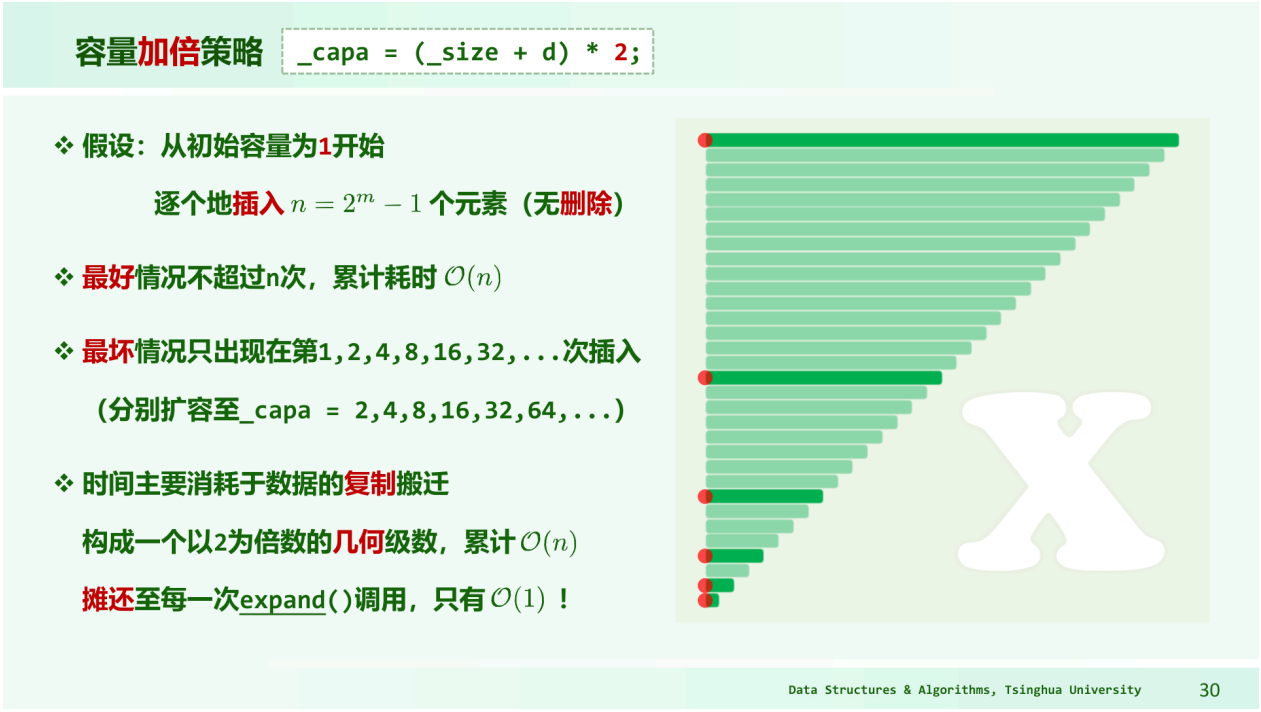


图 3　老师原页：向量章节中的成本分析。

5.2 一次扩容很贵，为什么尾插还能摊还常数

容量满时翻倍：复制 1、2、4、8.....个旧元素。做 n 次尾插，复制总量小于 2n，再加 n 次新元素写入，总量 O(n)，故每次摊还 O(1)。这不是说每次都只做一步；刚好触发扩容的那一次仍是 O(n)。

一般按固定比例 α>1 扩容，复制和为几何级数，仍 O(n)。α 越接近 1，复制常数越大；α 较大则空置空间更多。实际 `std::vector` 的增长比例不是由 C++ 标准统一规定的固定数字。

摊还平均的对象是整个合法操作序列，包括那些“不扩容”的便宜调用。若只拿真正扩容的调用做平均，就把为昂贵操作付账的便宜时刻删掉了。批量插入 k 项至少要 O(k) 写入，不能把“每调用一次”都许诺 O(1)。

5.3 为什么满了翻倍、四分之一再缩

如果一半就缩，一删一插可能导致缩容、扩容来回抖动。四分之一才缩到一半，为两种调整留出间隔，使大量普通操作夹在昂贵搬迁之间。势能法的核心是记录“先攒下、后支付”的信用，不是发明负时间。

5.4 有序去重：读写双指针

对 [1,1,2,2,2,4]，读指针顺序扫描，写指针指向已保留前缀的尾部。新值不同才写入，最终前缀为 [1,2,4]；时间 O(n)、额外空间 O(1)。空向量必须先处理，不能先读 a[0]。若等价元素还带不同附加信息，应明确保留第一份还是最后一份；课件的“稳定去重”有特定约定，别与排序稳定性混为一谈。

自测。 向量尾插摊还 O(1)，能否推出头插也摊还 O(1)？答：不能；每次头插仍要移动当前全部后缀，连续 n 次总量 Θ(n²)。

6 二分查找：先说清“找哪个边界”

6.1 找到一个、最左一个、最后一个，三个任务不同

本章采用半开区间 [lo,hi)。找第一个不小于 x 的位置，也叫 lower_bound：左侧全部 <x，右侧全部 ≥x，中间尚未确定。初始 lo=0、hi=n。

```
int lower_bound_index(const std::vector<int>& a, int x) {
    int lo = 0, hi = (int)a.size();
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] < x) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}
```

若答案等于 n，说明所有元素都小于 x，不能再读 a[n]。判断存在要额外检查 `p<n && a[p]==x`。返回插入位置与返回“某个相等项”不是同一种接口。

例：a=[1,3,3,7]，x=3。先 mid=2，a[2]≥3，hi=2；再 mid=1，hi=1；再 mid=0，lo=1；最后 [1,1)，返回 1。每一步宽度严格缩短，所以不会卡住。

6.2 Fibonacci 与插值查找

若向左和向右的比较成本不同，最优切分不必正好一半，Fibonacci 查找由此有意义。任意固定比例 $\alpha\in(0,1)$ 切分，只要两侧都严格缩小，最坏仍 O(log n)，常数不同。比例若随 n 靠近 0 或 1，则不能继续沿用固定比例结论。

插值查找利用值分布估位置：$m = lo + \lfloor (x - a_{lo})(hi - lo)/(a_{hi} - a_{lo}) \rfloor$，需处理分母 0、越界和中间乘法溢出。分布均匀等假设下可很快，畸形分布却可能退化为线性；它没有无条件优于二分的保证。很短的区间中顺序扫描还可能因缓存和分支开销更快，阈值应实测。

6.3 位图：一个可能值，分配一个 bit

整数范围 [0,U)，令第 x 位表示 x 是否出现。字节索引 x/8，位索引 x%8，用无符号掩码操作。空间 Θ(U) bit；扫描输入 O(n)，若按整个值域输出有序结果，还要 O(U)。所以总时间应写 O(n+U)，不能因为 n 很小就忽略巨大值域。

稀疏集合的校验环可用 dense 与 sparse：x 有效，当 `sparse[x]<size && dense[sparse[x]]==x`；插入同时维护反向关系，清空只令 size=0。它给 O(1) 逻辑重置，不表示底层数组不占 Θ(U) 空间，也不允许在 C++ 中读取从未合法初始化的任意标量来制造未定义行为。

自测。 二分代码中把 `hi=mid` 改成 `hi=mid-1` 能否照旧？答：不能；半开边界的不变量随之破坏，可能漏答案。边界和更新公式必须整套设计。

7 排序：除了排好，还要知道搬了多少次

7.1 稳定性，用带名字的相等项检查

输入 [(2,A),(1,B),(2,C)]，按数字排序。稳定结果应保持 A 在 C 前。只看 [1,2,2] 无法检测稳定性，必须给相等关键字附上原编号。

冒泡只交换相邻逆序项，使用严格大于比较时可稳定；一趟把最大项推到末尾。若一趟无交换即可提前结束，最好 Θ(n)，最坏 Θ(n²)。记录最后交换位置可跳过已确定后缀，但不改变最坏阶。

插入排序维持有序前缀，逐个把新元素插入其中。向量中的移动次数与逆序对数有关，时间可写 Θ(n+I)，I 为逆序对数。逆序输入 I=n(n-1)/2；互异元素均匀随机排列时，每对逆序概率 1/2，所以 $E[I] = n(n-1)/4$。

7.2 选择排序少移动，不代表少比较

每轮选最大项移到末端，比较数始终 Θ(n²)，但交换次数 O(n)。普通向量直接交换版本通常不稳定。链表若以节点转移而非任意交换实现，并妥善选择最后一个最大项等规则，可以保留相等项顺序；稳定性取决于具体实现，不是算法名字自动保证。

7.3 二路归并，谁先耗尽都要想

把 [1,4,6] 和 [2,3,5] 合并：比较两个队首，输出较小者并推进对应指针；得到 [1,2,3,4,5,6]。相等时先取左边可保持稳定。一个序列耗尽后，复制另一边剩余部分，成本 O(m+n)。若缓冲只复制左半，右半余项可能已经位于正确位置，要按代码的实际布局解释。

归并排序分两半、各自排好、再线性合并，O(n log n)。数组通常用 O(n) 辅助缓冲，可全程复用一次申请；链表可通过改链接合并，省元素缓冲，但递归栈仍要计算。看到“inplace_merge”也不能推断标准库保证任意实现都完全不申请额外空间，应查接口复杂度与可用缓冲条件。

7.4 归并顺手数逆序对

当右侧当前项比左侧当前项小，左侧尚未输出的所有项都与它构成跨区逆序。例如左 [2,5,8]、右当前 3，3 与 5、8 构成两对。累计左右递归内部逆序数，加上跨区数量，O(n log n) 完成；计数最大可达 n(n-1)/2，要选足够宽整数。

自测。 二分确定插入位置后，数组插入排序是不是 O(n log n)？答：比较可降到 O(n log n)，但搬移最坏仍 Θ(n²)，总时间不会因此自动降阶。

8 列表：改链接很快，找到位置可能很慢

8.1 把“已知节点”和“已知秩”分开

向量能 O(1) 按秩访问；列表通常要沿链接走 O(n)。双向列表已知节点 p 时，插在 p 附近或删除 p，可做常数次链接修改。但若先用值查 p，就还要加上查找成本。说“链表删除 O(1)”时必须带上“已知合法节点位置”的前提。

带头尾哨兵的空表也有 head→tail、tail→head，真实元素夹在中间。插入新节点 x 到 p 与 q 之间，要设置 x.pred=p、x.succ=q，再令 p.succ=x、q.pred=x。删除 x 前先保存其邻居，接好 p 与 q，再销毁 x；销毁后不再读取 x.succ。

8.2 单链表找倒数第 k 个

先让 fast 领先 k 步，再让 slow、fast 同速走；fast 到尾后位置时，slow 就在倒数第 k 个。要规定 k 从 1 开始，检查 k≤长度；k=0 的含义若未定义就应拒绝。两次遍历与双指针都是 O(n)，但实际读缓存、分支和距离不同，不能只靠“大 O 一样”预测秒数。

8.3 复制不是抄一根头指针

浅复制只复制节点地址，两份列表会共享同一批节点，随后析构可能双重释放。深复制要给每个元素新建节点并保持顺序。若从前到后读旧表却每次 insertFirst，新表会反转；想保序可从后往前读，或改 insertLast。

游标列表用数组下标代替指针，并维护空闲位置表，适合不方便动态分配或需要紧凑管理的场合。逻辑邻居不必是物理相邻数组格；这也说明“操作是否改变结构”要看逻辑内容，不能只看有没有重排物理地址。

8.4 列表上的排序取舍

插入排序可以通过摘下节点再接入合适位置，元素搬移常数，但定位最坏仍线性；归并排序顺序走链接，天然适配列表。二分查找需要快速到中点；列表没有随机访问，比较次数即使少了，走链接的成本仍可能线性，不能直接照搬向量结论。

自测。删除单链表尾节点，有尾指针是否就 O(1)？答：通常还需要找到前驱；没有前驱指针或额外结构时，仍可能 O(n)。删除节点后要正确更新尾指针与空表状态。

9 栈：最后放进去的，最先拿出来

9.1 先选对栈顶放哪

向量尾端作栈顶，push/pop 不必移动其他元素，push 摊还 O(1)，pop 通常 O(1)。若把头端当栈顶，每次进出都可能移动后缀，退化为 O(n)。双向列表两端都可常数操作，单向列表则通常更适合头端作栈顶。

括号匹配就是最直接的栈应用：左括号入栈，右括号必须与栈顶匹配并弹出。读完后栈必须空。输入 `(]` 虽然左右括号数量相同，却在 `)` 处发现栈顶是 `[`，因此失败。要报告位置，把括号与原位置一起压入；HTML 等格式还要先正确分词，不能把字符串、注释里的符号当普通括号。

9.2 表达式求值：减法要先拿右边

中缀 `3*(4+2)-5` 对应 RPN（逆波兰式）`3 4 2 + * 5 -`。读到数字压值栈，读到二元运算符先弹右操作数 b，再弹左操作数 a，计算 a op b 后压回。否则 `8 3 -` 会被错算成 3-8。

转换中缀时，运算符栈暂存尚未执行的操作；比当前运算符优先的栈顶先输出。同优先级还要看结合方向：减法通常左结合，乘方通常右结合，`2^3^2` 按数学惯例是 $2^{(3^2)}=512$，不是 $(2^3)^2=64$。注意 C/C++ 中 `^` 是按位异或，不是乘方。

阶乘是后缀一元运算，`!!` 若表示双阶乘，就需要在词法层把两个字符识别成一个 token，不能简单按“两个阶乘”执行。双阶乘 $6!!=6\cdot4\cdot2=48$，而 $(6!)!$ 完全是另一回事。除零、非法运算元和空栈都应作为可报告错误，而不是继续胡算。

9.3 栈混洗与合法括号

输入按 1、2、3 依次到来，允许入栈或弹出。输出 [2,1,3] 可行：压 1、压 2、弹 2、弹 1、压 3、弹 3；[3,1,2] 不可行：为了先弹 3，1、2 已在下面，下一项只能先是 2。

将一次入栈写 `(`，出栈写 `)`。任意前缀中入栈数≥出栈数，最终相等，正好是合法括号的条件。互异输入的栈混洗数为 Catalan 数

$$C_n=\frac{1}{n+1}\binom{2n}{n},\quad C_0=1,\quad C_n=\sum_{k=0}^{n-1}C_kC_{n-1-k}.$$

递推是按第一对匹配括号内部含 k 对来分类。字符重复时，不同操作序列可能得到同一个可见字符串，不能直接把 Catalan 当去重后的字符串数。

自测。 运算符种类只有十种，运算符栈能否最多十项？答：不能；任意深的括号嵌套和待处理操作可令栈随表达式长度线性增长。

10 队列与单调栈：什么时候才能放心丢掉一个候选

10.1 队列按到达顺序服务

队列先进先出。循环数组可让 head、tail 回绕，避免每次出队搬动后缀。区分满与空可额外记录 size，或牺牲一格容量；不能只看 head==tail 却同时表示两种状态。数组容量 N 时，索引更新为 `(i+1)%N`，还要确保 N>0。

双栈当队：入队压进 in；出队若 out 非空就弹 out，否则把 in 全倒进 out。单次倒腾可能 O(n)，但每项总共只进 in 一次、倒一次、出 out 一次，所以 m 次操作总 O(m)，摊还 O(1)。没有假定输入随机。

10.2 直方图最大矩形，画边界再写栈

高度 [2,1,5,6,2,3]，最大矩形是中间高度至少 5 的两根柱，面积 5×2=10。对每根柱，找左右第一个比它低的柱，二者之间都不矮于它，因此它作为最矮高度时的最大宽度就是两个边界之间的柱数。

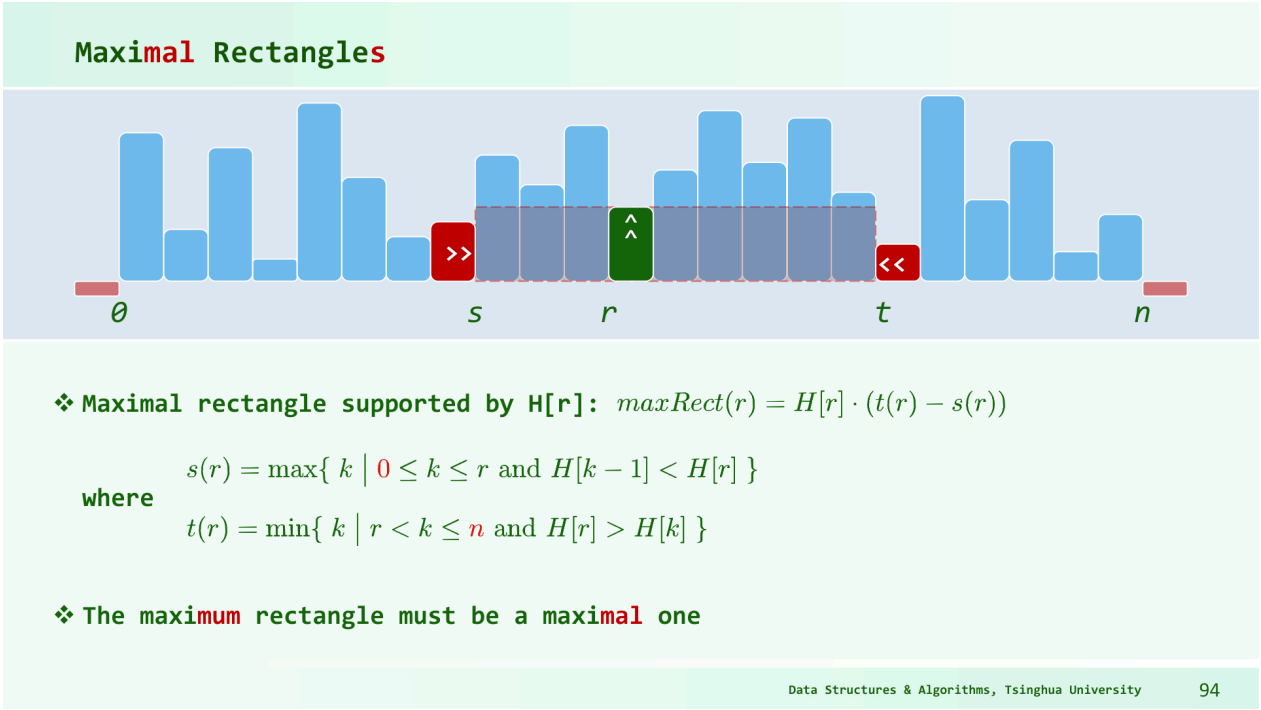


图 4　老师原页：直方图问题与边界结构。

单趟递增栈保存仍有希望向右延伸的柱下标。新柱更低时，栈顶柱的右边界被确定，就弹出并算面积；弹出后新栈顶给出左侧边界。末尾加一根虚拟高度 0，或单独清栈，处理所有未结算项。相等高度要统一采用严格/非严格比较约定，否则左右边界和并列答案会不一致。

每根柱入栈一次、出栈最多一次，故嵌套 while 的总成本 O(n)，辅助空间最坏 O(n)。不是“for n 次乘以 while n 次”得到 n^2；同一根柱不可能反复出栈。

10.3 最大队列与滑动窗口

维护一个普通队列保存全部窗口元素，再用单调双端队列保存最大值候选。新值 v 到来，把尾部所有比 v 小、且会更早离开窗口的候选删掉：它们以后既不比 v 大，也活不过 v，因此永远不会再成为最大值。队首就是当前最大。

重复值最好保存下标或计数，过期时按实际身份删除。窗口 [3,3] 删除第一份 3 后，第二份仍有效，不能因为值相等就把所有 3 都删掉。

10.4 期望与最大值不能偷偷交换

某一步栈大小的期望小，不代表整个运行期间峰值的期望等于它。一般 $E[\max_i X_i] \neq \max_i E[X_i]$。若题目要求期望辅助空间，还要明确输入是随机排列、独立连续高度，还是其他分布；没有概率模型就不能随口说 O(log n)。

自测。 单调队列出队时为什么只从头检查过期？答：候选按时间顺序排列，最旧项一定在前面；若用值而不带位置，重复元素会使这一逻辑含糊。

11 二叉树：每一棵子树都还是同一种问题

11.1 先统一高度的定义

本书路径长度按边数计，根深度为 0，叶高度为 0，空树高度为-1。节点高度等于 $1+\max(h_L,h_R)$。若有 n 个节点的非空树，边数 n-1；树高为 h 时最多 $2^{h+1}-1$ 个节点，所以 $h \geq \lceil \log_2(n+1) \rceil - 1$。

设二叉树叶节点 n_0、单孩子节点 n_1、双孩子节点 n_2。按节点数算边为 $n_0+n_1+n_2-1$；按孩子数算为 n_1+2n_2；两式相等得 $n_0 = n_2 + 1$。这条对所有非空二叉树成立，不必先假设没有单孩子节点。

11.2 三种遍历，只看“什么时候访问根”

对根 A、左子 B、右子 C：先序 A B C，中序 B A C，后序 B C A。先序适合先处理父再处理子；后序适合子结果先算好，再汇总到父，比如计算高度或安全释放整棵树。

迭代中序：沿左链把节点压栈，左边走尽后弹一个访问，再转向其右子树。栈记录“左边处理完后，还要回来做的事”。每节点入出各一次，时间 O(n)，辅助栈 O(h)。层次遍历用队列，队列内节点来自相邻层，空间与最大层宽同阶，最坏 O(n)。

11.3 中序后继怎么找

有右子树：后继是右子树最左节点；没有右子树：沿 parent 向上，直到第一次从某祖先的左子树返回，该祖先就是后继；若一直从右侧返回至根外，则不存在后继。最后一个中序节点的后继必须是空，而不是绕回根。

11.4 哪些遍历能重建

节点标签互异时，先序+中序、后序+中序能唯一重建：根在先序首或后序尾，再用中序位置把左右规模分开。普通二叉树先序+后序通常不唯一，例如只有 A、B 两点，B 是左孩子或右孩子，序列都相同。若限制为真二叉树（每个内部节点恰有两个孩子），歧义可消除。

加入空指针标记的先序序列能重建一般二叉树：读一个标记，空则返回，否则建节点，再递归读左右。只给中序空标记仍不能一般唯一重建，不能把所有“增强序列”都看成一样。

多叉树可用左孩子右兄弟表示：左指针连第一个孩子，右指针连下一个兄弟。对有序森林，也可把各根当兄弟串起来。此时二叉边中的“右边”不再表示原树的孩子，解释深度时要回到原语义。

自测。删除一片叶子后，某祖先高度没变，更高祖先还要更新高度吗？答：若其他结构不变且只维护由孩子高度定义的高度，更高处也不变，可停止；若还维护 size 等属性，不能一起停止。

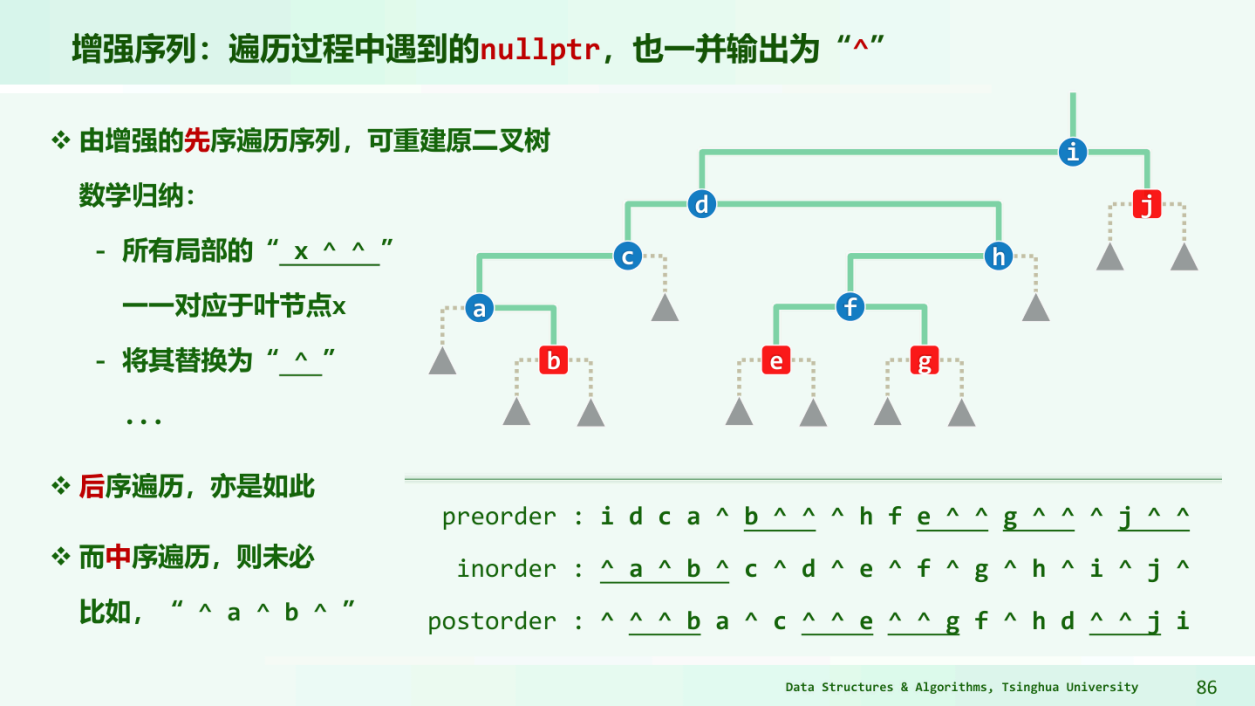


图 5　教师原图：带空标记的三种遍历。

图里每个 ^ 表示一个空孩子。按先序读到真实标签就建节点，再依次读取两个孩子；空标记让“这个孩子到底不存在，还是还没读到”不再含糊。中序的 ^ a ^ b ^ 反例说明，标记本身还不够，读取次序也很重要。

12 Huffman：让常见字符少走几步

12.1 编码成本是“频率乘路径长度”

前缀码不让一个字符编码成为另一个的前缀，因而可从根沿 0/1 边走到叶子后立即确定一个字符。权重 w_i、编码长度 ℓ_i，总成本是 $\sum w_i\ell_i$，不是所有路径长度简单相加。

例：A:5、B:2、C:1、D:1。先合并 1、1 得 2；再合并 2、2 得 4；最后合并 4、5 得 9。A 深度 1，B 深度 2，C、D 深度 3，总成本 $5+4+3+3=15$。左右 0/1 可互换，树也可能因同权选择不同，但最优成本不变。

12.2 为什么最小两项可以先合

在某棵最优前缀树中，可把权重最小的两项交换到最深的一对兄弟叶子，不增加成本：若较大权重在更深处，交换后成本变化为 $(w_{大}-w_{小})(d_{浅}-d_{深})\le 0$。把这对兄弟缩成权重和为 w_1+w_2 的一个叶子，剩下就是规模减一的同类问题。否则若缩小问题能进一步优化，展开后原树也会更优，和原本最优矛盾。这给出贪心选择和最优子结构。

用最小堆反复取两个最小项，O(n log n)；若输入权重已排序，可用两条单调队列在线性时间完成合并阶段。先排序的成本不能隐藏，所以“总是 O(n)”是错的。编码树形成后，生成全部码字还受输出总长度影响。

12.3 排序为什么不能只做 O(n) 次比较

n 个互异元素有 n! 种可能顺序，每次二元比较至多把候选分两支。比较决策树若所有叶深都小于 $\log_2(n!)$，叶子就不够，因此最坏至少 $\Omega(\log(n!))=\Omega(n\log n)$。均匀随机排列下，平均深度也受叶数与树结构约束，不能因为允许少数输入很慢，就让全部平均变线性。

这只限制比较模型。计数排序、基数排序借助关键字结构与取值范围，不只用两两比较，所以不与下界冲突。Huffman 构造问题的下界也要说清输入是否预先排序、是否要求输出有序叶序等模型；不要把“采用堆的实现是 O(n log n)”误当所有变体的绝对下界证明。

自测。 最常见字符一定编码为 0 吗？答：不一定；它通常应更短，但 0/1 方向可整体交换。评价的是长度和可解码性，不是某个字符必须用哪个位串。



图 6　教师原图：Huffman 相关结构与编码推导。

读图时给每个叶子标频率，给每条边标 0/1；走到叶子后把沿途位串写下来。最后逐项核对频率乘深度的总和，而不是只看树是否长得对称。

13 搜索树：别让一条长链假装成一棵树

13.1 BST 的规则是对子树说的

每个节点左子树关键字小于它，右子树关键字大于它，重复键须另定策略。中序遍历因此有序。查找每次比较决定向左或向右，代价 O(h)。若依次插入 1、2、3、4 到普通 BST，会形成单链，h=n-1，查找就退化为 O(n)。

删除分三种：无孩子直接摘除；一个孩子用孩子接替；两个孩子可找中序后继交换/复制记录，再删除后继节点。若记录中包含外部句柄或身份约束，交换数据与交换节点并不总是等价，要按接口约定决定。

13.2 AVL 平衡的是高度差

要求每个节点左右子树高度差不超过 1。最少节点数满足 $N(h) = 1 + N(h-1) + N(h-2)$，至少按 Fibonacci 速度增长，因此高度 O(log n)。这里不是“看起来左右差不多”，而是每个节点都必须满足可检查条件。

插入导致某祖先失衡，按路径形状分 LL、RR、LR、RL。LL 用右旋，RR 用左旋；LR 先对子做左旋，再对祖先右旋；RL 对称。旋转保持中序顺序，只改变局部高度：节点之间夹着的那棵子树，关键字范围仍在两者之间，接到对应空位即可。

例：依次插入 30、10、20。30 的左子树过高，但新增点在其左孩子的右侧，所以是 LR：先围绕 10 左旋，得到 20 在 10 之上；再围绕 30 右旋，得到根 20，左右 10、30。

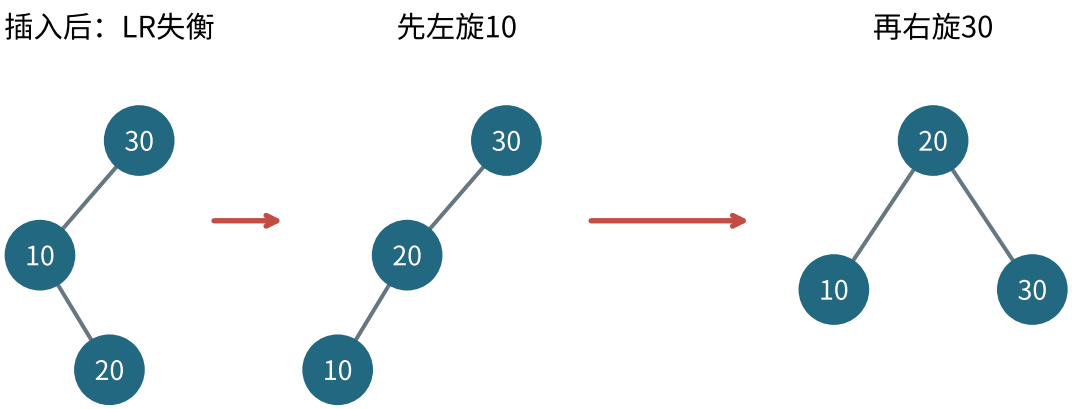


图 7　本书原创图：30、10、20 的 LR 双旋，按两次独立旋转看。

13.3 插入与删除不能完全照搬

插入在第一次失衡处修复后，局部高度通常恢复到插入前，更高祖先不再因这次插入继续失衡。删除修复后，局部高度仍可能降低，影响更高祖先，因此可能一路修到根。旋转后要按自底向上顺序更新高度和其他增广信息。

自测。一个节点左右孩子关键字满足大小关系，是否就保证整棵是 BST？答：不保证；需要整个左/右子树的所有关键字满足范围。验证可递归传上下界，不能只比较一层孩子。



图 8　教师原图：局部旋转保持中序。

看左边的 zig：c 到上面，v 到右边，中间子树 C 改接到 v 左边。C 里的键仍在 c 与 v 之间，所以中序未变；只改常数根链接。之后重新算低处节点高度，再算高处节点高度，顺序不能颠倒。

14 高级搜索结构：不同方式把路径缩短

14.1 伸展树：把刚用过的搬到上面

每次访问后将目标或最后访问节点旋到根附近。单步 zig、同向 zig-zig、异向 zig-zag 的组合调整结构。它不保证每次 O(log n)，一条操作可能 O(n)，但在相应分析与初始条件下，长操作序列的摊还成本可达 O(log n)。热点集中时有自适应优势。不要把摊还结论改写成最坏单次保证。

14.2 B 树：一次读磁盘块，多比较几个键

一个节点放多个有序键和多个孩子，减少树高与外存 I/O。以最小度数 t 的定义为例：除根外每节点至少 t-1 个键、至多 2t-1 个键；内部节点孩子数等于键数加 1；所有叶在同一层。另有“m 阶 B 树”的记法，含义不同，先核对教材定义再代数。

插入到满节点时分裂，中间键上升到父；删除时若孩子即将过少，先向兄弟借键或合并。叶深相同的约束不能破坏。例：t=2，节点 [10,20,30] 分裂，20 上升，左右为 [10]、[30]。目标是减少外存访问，不只是减少 CPU 比较次数。

14.3 红黑树：给路径一个颜色约束

常见定义：节点红或黑，根黑，空叶黑，红节点不能有红孩子，从任一节点到后代空叶的黑节点数相同。任一路径中红节点至多与黑节点交替，最长路径至多约为最短的两倍，因此高度 O(log n)。插入和删除靠重新着色与旋转修复，不应把“每次只转一两下”当删除全部过程的描述。

理解修复时先画叔叔、兄弟和侄子节点的颜色，标清缺失黑高度在哪侧，再做变换。颜色约束是证明高度的工具，不是键值大小的替代；旋转仍须保持 BST 次序。

14.4 跳转表：用随机楼层跳过一大段

底层包含所有有序项，每个节点以固定概率 p 升到上一层。查找从高层向右，不能再前进就下降一层。适当随机模型下，期望查找/插入/删除 O(log n)，期望空间 O(n)，最坏仍可能很差。随机性是建结构时的假设，不是输入“看着乱”就足够。

自测。AVL、红黑、伸展、跳转表都能查找，为什么不只学一种？答：它们对严格最坏界、摊还成本、局部性、实现复杂度和并发支持有不同取舍，应用需求不同。

15 搜索树应用：一个点能找到，一大片范围怎么找

15.1 给树增加能快速汇总的信息

在 BST 节点保存子树大小，就能按秩选第 k 小：左子树有 L 项，k=L+1 时当前就是答案，k≤L 就向左，否则向右查第 k-L-1 小。旋转后也必须更新 size，否则树形正确、查询结果仍会错。

区间树常把区间按一个端点组织，并保存子树最大右端点。查询与 [a,b] 重叠的区间时，可据此剪去不可能有答案的分支。先明确闭区间重叠条件为 $l \le b \wedge a \le r$；半开区间则要改严格性，端点恰好相接是否算重叠不能混。

15.2 线段树：把一段区间分成少数块

把 [0,n) 反复二分，每节点保存对应段的和、最大值等可合并摘要。单点更新沿一条根到叶路径 O(log n)；区间查询把目标拆成 O(log n) 个规范节点，合并摘要。存储常见 O(n)，数组实现经常预留约 4n 是实现技巧，不是精确数学节点数。

例 a=[2,1,3,4]，查 [1,4) 的和。根分 [0,2)、[2,4)；左边再取 [1,2) 得 1，右边整段得 7，总和 8。若范围整段加 v，节点保存懒标记，并把区间和加 $v \times$ **长度**；往下访问前下推标记。最大值摘要则加 v，不乘长度。

15.3 范围树和 kd 树，解决多维限制

二维范围查询要找同时满足 x 和 y 区间的点。范围树常按 x 建平衡树，在规范子树中配 y 结构，经典二维正交报告可得 O(log²n+k) 查询、O(n log n) 空间，k 是输出数；增强方案可进一步改进，不能不说明变体就混写界。

kd 树轮流按维度划分点集，节点对应空间区域。查询先看区域与查询框：完全不相交就剪枝，完全包含则整片报告，部分相交再递归。平衡、固定维度等前提下，二维正交报告有经典 O(√n+k) 界；最近邻查询在有利分布常很快，但最坏可能访问很多节点。

自测。 要输出 k 个点，查询能否无条件 O(log n)？答：至少需要 Ω(k) 的输出工作。复杂度必须把输出规模写出来。

16 散列：先把键分到格子里，撞车再处理

16.1 相同哈希值不等于相同键

散列函数 h 把键映射到表位置。不同键可能得到同一位置，这叫冲突；最终仍须比较完整键来确认相等。若表有 m 个桶、n 个元素，装填因子 $\alpha = n/m$ 衡量拥挤程度。

链地址法每桶挂一条链，适当均匀散列假设下，查找期望 O(1+α)。这不是最坏 O(1)：所有键落同一桶时会退化。表太满就扩容并重新计算位置，因为 m 变了，旧桶号通常不能原封不动搬过去。

16.2 开放定址：表内继续找位置

线性探测依次检查 h(k)、h(k)+1……，容易形成聚集；平方探测和双重散列改变探测序列，各自需要满足表长与步长等条件，才能保证探测覆盖与正确性。查找必须沿插入时同一条序列走，不能随便换一个探测公式。

删除不能直接把槽清成“从未使用”。假设 A、B 冲突，B 被放到后一个位置；删 A 后查 B，若见空就停止，会误报不存在。应使用删除标记，或采用能够保持探测链正确性的回移删除方案。删除标记多了，也要适时整理重散列。

16.3 一个可手算的小表

m=7，h(k)=k mod7，线性探测插入 10、17、24：依次落在 3、4、5。查 24 要依次看 3、4、5；删 17 留下墓碑后，查 24 仍须经过 4 继续；新插 31 可利用合适的墓碑位置，但还要避免已有同键未被发现的重复插入。

16.4 散列并非排序

哈希表擅长按键相等查询，通常不直接提供按序遍历、前驱、范围报告。若题目要“所有位于[100,200]的键”，普通哈希表可能要扫描整个表；平衡搜索树可能更合适。

自测。随机散列和随机输入是一回事吗？答：不是。可对固定输入随机选函数来分析期望；也可固定函数假设输入分布。必须说明究竟随机的是谁。

17 图基础：先记录访问过谁，再沿边走

17.1 表示方式决定很多成本

邻接矩阵用 $\Theta(V^2)$ 空间，查一条指定边 O(1)，遍历某点邻居需 O(V)；邻接表用 $\Theta(V+E)$ 空间，遍历邻居与度数相关。无向图的边通常存两次，所以遍历总邻接项是 2E，仍为 O(E)。自环、重边和有向性要在输入约定里写清。

17.2 BFS 按距离一层一层扩张

从源点入队时立即标记已发现，弹出后扫描邻居，未发现的设距离加一、记录父节点并入队。必须在入队时标记，否则同一节点可能被多条边重复加入。无权图或等权图中，先发现的路径就是最少边数路径；一般带权图不能这样直接算最短路。

对 A-B、A-C、B-D、C-D，BFS 从 A 得距离 0、1、1、2；D 第一次由 B 或 C 发现都可得到最短距离，父树可能不同。用邻接表，时间 O(V+E)，队列最坏 O(V)。不连通图要从每个尚未访问点重新启动，得到森林。

17.3 DFS 记录“进入”和“退出”

DFS 沿一条路深入，走不下去才返回。发现时间与完成时间可帮助判断祖先关系和边类型。有向图中遇到仍在递归栈内的点，说明存在环；仅“访问过”还不够，因为对已完成节点的边未必构成当前回路。

DAG 的拓扑排序可用入度队列：初始压入全部入度 0 点，删除它们及出边，使新入度 0 点入队；若输出少于 V，说明有环。也可用 DFS 完成次序逆序，但同样需要检测环。

17.4 无向图的 low 值在说什么

disc[u] 是首次发现时间；low[u] 是 u 的子树通过树边下行、再至多一条回边能到达的最早祖先时间。非根 u 若存在孩子 v 满足 low[v]≥disc[u]，u 是割点；根要单独判断是否至少有两个 DFS 孩子。树边(u,v)若 low[v]>disc[u]，它是桥。≥ 和 > 不能交换。

用边栈在 low[v]≥disc[u] 时弹出相应边，可得到点双连通分量。无向图需要排除返回父边，但有重边时不能简单排除所有“指向父节点”的边，应按边 ID 区分。

自测。DFS 树根只有一个孩子，但孩子下面很多点，根是否一定是割点？答：不是；删根后其余节点仍可能都在那一棵子树内连通。根的判断规则与非根不同。

18 图进阶：最短的路和最省的连线不是同一个目标

18.1 松弛：发现一条更便宜的到达方式

距离估计 d[v] 表示当前知道的最短路径长度。沿边(u,v)检查 `d[u]+w(u,v)<d[v]`，若成立就更新 d[v] 并记录前驱。无穷值参与加法前要检查，避免整数溢出把“不可达”变成负数。

Dijkstra 每次选未确定节点中 d 最小的一个，确认它，再松弛出边。正确性依赖边权非负：若以后绕一条路回来，额外边不会凭空把距离降得更低。零权允许，负权则不能沿用保证。

例：A→B=4，A→C=1，C→B=2，B→D=1，C→D=5。先确定 A；得到 B4、C1；再定 C，更新 B3、D6；再定 B，更新 D4；最后定 D。最短路 A-C-B-D 长 4。用二叉堆与邻接表，常见 O((V+E)log V)；若用重复入堆法，要跳过已过时的距离项。

18.2 最小生成树只求全网总成本最少

无向连通带权图的 MST 有 V-1 条边，连接所有点且无环。它不保证从某源到各点的路径最短。Prim 维护已选点集合，每次加入跨割的最轻边；Kruskal 将边按权排序，若两端属于不同连通块就加入，并用并查集维护连通性。

割性质给出贪心依据：对某个割，如果一条边是安全条件下的最轻跨边，可找到包含它的最小生成树。证明常用交换：若某最优树没有它，加入后形成环，再删掉环上一条不更轻的跨割边，总成本不会增加。

不连通时只能得到最小生成森林；负边权对 MST 没有 Dijkstra 那种障碍。相同权重时最优树可不唯一，总权重仍相同。不要为了得到“唯一答案”擅自改变问题定义。

18.3 并查集的两种优化

按秩/大小合并避免过深，路径压缩让查询经过的节点直接接近根。二者结合的摊还界是极慢增长的反 Ackermann 量级，但这不表示每一步严格常数。合并前必须先找两端根，不能随手把任意节点 parent 改成另一个任意节点。

自测。最短路径树是否一定是 MST？答：不是。前者分别优化从源点出发的路，后者优化所有选边的总和。画一个三点带权三角形就能构造反例。

19 优先级队列：每次先取最重要的，不必把全部排好

19.1 堆不是有序数组

最小二叉堆只保证父节点≤孩子，兄弟之间、两棵子树之间不保证全局有序。完全二叉树可紧凑放进数组：零基下标 i 的孩子是 2i+1、2i+2，父是 floor((i-1)/2)。根是最小项，但中序遍历不一定有序。

插入先放末尾，再沿父链上滤；删最小值时拿最后一项补根，再与更小的孩子交换下滤；两者最坏 O(log n)，看最小 O(1)。每次交换应同步更新外部位置索引，否则 decrease-key 等接口可能找错元素。

19.2 建堆为什么不是 n log n

自底向上，对最后一个内部节点开始依次下滤。高度为 k 的节点约不超过 n/2^{k+1}，每个最多移动 k 层，总成本

$$O\left(n\sum_{k\ge 0}\frac{k}{2^k}\right)=O(n).$$

大多数节点靠近叶子，根虽然要下滤很远，却只有一个。把每个节点都按树高 log n 收费，会得到不紧的 O(n log n) 上界。

19.3 锦标赛树、多叉堆与左式堆

锦标赛树保存两两比较的胜者，最优项在根；更换一个参赛者只更新它到根的路径，适合多路归并。败者树保存失败者信息，是相关但不同的组织方式。

d 叉堆把树高降为 log_d n，但每次下滤要找 d 个孩子中最优者，比较总量约与 d log_d n 有关；不能只因为树更矮就断言越多叉越快。

左式堆着眼于快速合并，维护左孩子的零路径长度不小于右孩子。合并沿右脊递归，再必要时交换孩子，保证右脊短。它不要求整棵树像 AVL 一样左右高度接近；“左式”约束与普通高度平衡不同。

自测。堆排序稳定吗？答：常见原地二叉堆排序不稳定，因为远距离交换可能颠倒相等关键字的原顺序。需要稳定性时必须额外设计，而不是换成 <= 就完事。

20 字符串匹配：已经比较过的信息，别白白扔掉

20.1 暴力匹配为什么可能重复做很多事

文本 T 长 n，模式 P 长 m。暴力法在每个位置从头比较，最坏 O(nm)。例如很多重复的 A，模式末尾才与文本不同，前面长前缀会在相邻起点反复比较。

20.2 KMP 的前缀表，先算一个小例子

π[i] 表示 P[0..i] 的最长真前缀，同时也是后缀的长度；“真”表示不能把整个串算进去。P=ABABAC 的 π 为 [0,0,1,2,3,0]。到第 4 项（从 0 计）ABABA，最长首尾相同是 ABA，所以值为 3。

扫描文本时，j 表示已经匹配的模式前缀长度。下个字符不匹配，就令 j=π[j-1]，尝试更短的首尾相同部分，而不把文本位置退回。j=0 仍不匹配时才跳过该文本字符；j=m 时报告一次匹配，再按 π[m-1] 回退以寻找重叠匹配。

建表 O(m)，扫描 O(n)：文本指针从不回退，j 的增长次数受文本长度限制，下降总量也不会超过增长加常数。空模式的返回语义应先规定，不能读 P[0] 越界。

20.3 Boyer–Moore 的两种跳法

从模式右端向左比较；坏字符规则根据不匹配字符在模式中的位置决定移动；好后缀规则利用已经匹配的模式后缀，寻找另一次出现或可对齐的前缀。两者取能保证安全的位移，不是想跳多远就跳多远。不同 BM 变体的最坏界不同，只有明确实现与附加技巧后，才可给严格线性保证。

20.4 KR 与键树

Karp–Rabin 用滚动哈希快速比较窗口指纹，但相同指纹可能碰撞。要确定性正确，就对哈希命中再核对原串；双哈希可降低碰撞概率，却不是数学上的“绝不会碰撞”。

Trie 按字符沿边走，查找与键长度有关；空间取决于分支表示和总前缀数。压缩 Trie 把没有分叉的路径合成边，减少节点；字符集大时不能默认给每节点分配完整巨大数组。

自测。 文本 AAAAA 中模式 AAA 有几处匹配？答：3 处，起点 0、1、2。报告匹配后若直接 j=0，可能错过重叠，需要按前缀表继续。

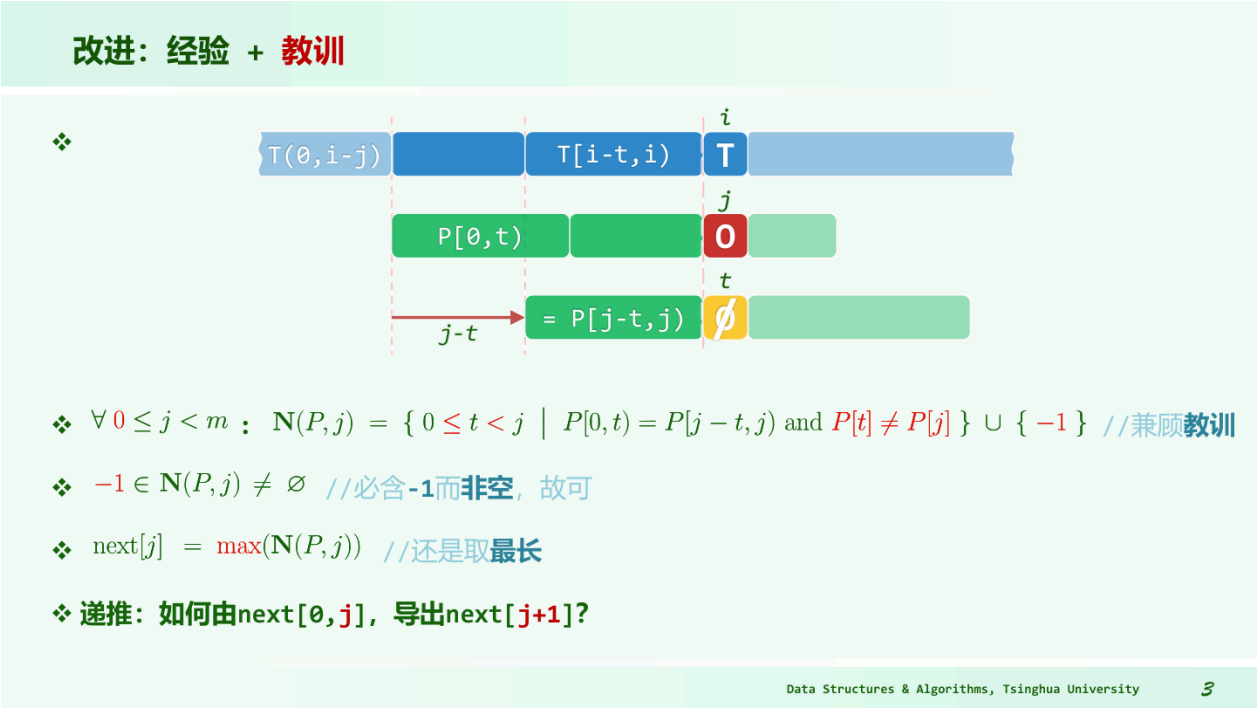


图 9　教师原图：改进 next 表排除下一步仍会失败的候选。

注意图中使用的是带额外条件 `P[t] != P[j]` 的改进失配表。正文的 π 表记录普通最长真前后缀长度，二者索引与定义不同。改进表会继续跳过已知比较同一失败字符的候选，因此不能把这张 next 表直接抄进使用 π 更新的代码。

21 排序与选取：只找第 k 小，不必把所有人排队

21.1 快速排序真正依赖的是划分

选轴点，把比它小、相等、大的元素分开，再递归较小和较大部分。平均或随机化期望 O(n log n)，不利轴点连续造成 1 与 n-1 划分时最坏 $O(n^2)$。全相等数据需要特别处理；三路划分可一次归拢相等区，避免做无用递归。

要控制递归栈，可先递归较小一侧，较大一侧改循环；这样即使总比较很坏，额外调用深度仍能控制为 O(log n)。直接两侧递归则最坏栈深 O(n)。比较器还必须满足一致的严格弱序，不能把 <= 当普通排序谓词。

21.2 Quickselect 只走答案所在一侧

完成一次划分后，若 k 落在小于区，就只递归那里；落在相等区就结束；否则调整 k 后进入大于区。随机轴点期望 O(n)，最坏 $O(n^2)$。取中位数的中位数可以建立最坏线性界：五个一组取中位数，再递归找这些中位数的中位数，保证每轮丢掉固定比例候选；证明和常数成本都比随机版复杂。

多数元素（出现次数严格超过 n/2）可用抵消思想找候选：不同值成对消去，真正多数不会被全部消光。但若题目不保证存在多数，最后必须再数一次候选频次确认。选出中位数也不自动证明它是多数。

21.3 希尔排序：先让远处元素靠近该去的位置

按一串递减 gap，对每个间隔子序列做插入排序，最后 gap=1。复杂度依赖 gap 序列，没有一个对所有版本都成立的“O(n log n)”统一结论。它通常不稳定，且经验效果与实现、数据有关。写考试答案时要注明所用增量序列，而不是只写算法名。

21.4 一张选用表

目标	常用方法	必须记住的条件
稳定全排序	归并排序	常用数组版要辅助空间
最坏 O(n log n)、原地	堆排序	常见版不稳定
平均很快全排序	随机快排/内省排序	纯快排最坏可二次
第 k 小	Quickselect	随机版只有期望线性
小整数范围	计数/基数排序	要计值域、位数与空间

自测。只需最小 10 项，n 很大，必须完整排序吗？答：不必；可用大小 10 的最大堆扫描，时间 O(n log10)，再把最终 10 项排序。若还要输出顺序或稳定规则，应另行处理。

22 PA1 五个主题：把讲解里的省略补出来

本章依据现有 PA1 讲解 PDF 的 Filename、Interview、Gift、Graphics，以及 PPTX 的二维偏序主题。当前 CLI 的课程资源清单可读，但作业枚举仍报告 `LEARN_HOMEWORK_ENUMERATION_UNVERIFIED`；因此不能把“取到了全部教师文件”夸成“已证明 OJ 全部 PA/Lab 题目无遗漏”。具体限制以各题正式题面为准。本章给独立小例子和推理，不复制个人提交。

22.1 Filename：插删距离与带状动态规划

若只允许插入与删除，把长度 n、m 的两串变成相同序列，保留下来的公共子序列越长，删除/插入越少。若 L 是 LCS 长度，最少编辑数是 $n+m-2L$：从第一串删 n-L 个，从第二串补 m-L 个。若允许一次替换且成本 1，这个式子一般就不再等价；先核对操作集合。

例：`ABC` 与 `ADC`，LCS=`AC`，插删距离 2。若预算 k=1，无解；若允许替换 B→D，则只需 1，说明问题定义会改变答案。

令 D[i][j] 为插删距离，递推取删、插、字符相等时对角转移。若目标距离≤k，则路径任何状态都应有 $|i-j| \le k$，因为长度差本身就至少需要这么多次插删。只计算对角线两侧宽 k 的带，状态 O((n+m)(k+1))，滚动行空间 O(k+1)。还可先用 |n-m|>k 剪枝。教师讲解有一处用“大于 k”的文字描述带内有效目标，需结合上下文更正：我们保留的是**可能达到距离不超过 k 的状态**。

22.2 Interview：按顺序数第 k 个空位

逆序恢复过程后，需要在环上寻找第 k 个空座位。朴素逐位走每次可能 O(n)。分块每块约√n 项，记录空位计数，先整块跳过，再在目标块内扫描，单次 O(√n)，并不是无条件 O(1)。讲解的引导句和随后复杂度表述不一致，应以后面的分块实际成本为准。

用一个 8 座位小例子：空位下标 [0,1,3,4,7]，从 1 之后数第 3 个空位，依次 3、4、7，落 7；若从当前座位起计就会得到另一答案，所以起点是否计数必须按题面固定。树状数组或增广树可把“第 k 个 1”查询降到 O(log n)，但需证明环绕和秩转换。

22.3 Gift：中间相遇，把指数砍成两半

每组二选一，总共 2^n 种。拆成左右两半，分别列出所有半和 L、R。对每个右半和 r，统计左半和≤P-r 的数量。把 L 排序后上界二分，复杂度常见 O(n·2^{n/2})，存储 O(2^{n/2})；这里 n 因子来自枚举方式或排序的 log，不应随便丢。

例：两组(1,4)、(2,5)，预算 6，组合和 3、6、6、9，合法 3 种。相同和代表不同选法，不能去重后按“不同和数”算方案数。

若每次把已有有序半和整体加 a、整体加 b，再线性归并，两列各自仍有序，可在线性总输出量内生成有序列表；左右都有序后用双指针一次统计。总状态量仍指数，只是从 2^n 缩到约 2^{n/2}，并非变成多项式。计数和总和需防溢出。

22.4 Graphics：先证明比较结果会单调

排序后的分界线若满足不交叉等题面结构，可用叉积判断点在线哪侧，再二分定位区域。对向量 $u=(u_x,u_y)$、$v=(v_x,v_y)$，叉积标量为 $u_xv_y-u_yv_x$；正负表示有向旋转方向。二分前必须证明分界线按顺序给出的谓词是“先真后假”或相反。若线段相交、点在边上却未规定归属，直接套二分就可能错。

大坐标相乘的中间结果可能超过 64 位，即使最终差值很小；应按题面上界选宽度，必要时用受支持的更宽整数或精确比较方法。

22.5 二维偏序：排序以后还要处理相等

对三元组(a,b,c)，希望沿 a、b 都不减的链，最大化 c 总和。若题目允许任意选择顺序、等坐标可串接且不要求至少取一项，在不改变可行性的前提下可删负权，等(a,b)合并权重。若要求非空或题目另有限制，不能无条件删除全部非正项。

先按(a,b)字典序排，再对前面 b≤当前 b 的最佳值做 DP：$f_i = c_i + \max(0, \max_{j<i, b_j \le b_i} f_j)$。坐标压缩配前缀最大值树状数组可 O(n log n)；需要严格小于时查询边界和相同 a 批处理都要改。例：(1,3,4)、(2,2,5)、(2,4,6)，前两项互不可比，第三项可接任一，最佳为 5+6=11。

讲解 PPT 的分治也在维护同一转移：先解左半，把左到右的贡献算完，再解右半。若像普通归并那样先把右半全部解完，再传左边信息，右边内部依赖可能来不及重新传播。CDQ 的执行顺序本身就是正确性的一部分。

自测。“两个坐标都不减”是否等于“按两个坐标排序后全选”？答：不是；(1,3)与(2,2)在字典序中有先后，却不能同时形成不减链。

23 PS01：算法、级数与递归，32 题逐项检查

打开教师原始习题单。下面按原题号逐项说明。符号有歧义时，以原题所用接口、区间与概率模型为准；测试/演示题给出可以执行的检查方法，不伪造已经完成的硬件实测。

23.1 1 · 计算工具

可选算盘加法、尺规作图、机械分拣。每个例子都写出输入怎样编码、允许哪些动作、结果怎样读出。C++使用离散符号执行；尺规中的几何对象与精确构造是另一套基本操作，不能直接比较“指令条数”。

23.2 2 · 曹冲与刻舟

水位对应排水重量，替换石块时仍保持同一水位；船移后船上刻线却不再对应落剑的地面位置。写出状态变化前后被保持的量，说明算法正确性依赖哪项物理假设。

23.3 3 · 终止与死循环

每个循环列一个非负整数下降量，检查溢出和边界是否会让下降失效。通用自动判停不可能，反证见正文 01；有限程序集合、受限语言仍可有专门验证器，不能由“不可能通用”推出“一切验证都无用”。

23.4 4 · Failstone

这道题不能把“像 Hailstone”当尚未解决。按原式，p>q 时变(p-q,p)；p<q 且 q 偶时 q 减半；q 奇时 p 翻倍。终止证明：在 p<q、q 奇的阶段，有限次翻倍到 p=小于 q、但 $2p$>q；再翻倍、相减换位、把偶数第二项减半，得到(2p-q,p)，两项都小于原 q。q 偶时直接让最大值下降；p>q 时先一步转为较小项在前，再进入前述阶段。每个有限宏步骤使正整数 max(p,q)严格下降，最终只能 p=q。程序计数须把每个原递归分支的“加 1”都计入，不能只数宏步骤。

23.5 5 · 几何分布

独立投硬币直到首次正面、固定命中率下重试，都符合模型。写 P(K=k)=(1-p)^(k-1)p，期望 1/p；网络重试若命中概率随时间变，或事件相关，就不能不加说明套公式。

23.6 6 · 素因子分解性能

最坏可选接近上界的素数，按原题一直试到因子自身的朴素流程会检查大量素数；最佳可选小素数的幂，反复除同一个因子。若自行加入“只试至平方根”的改进，要明确已改变算法，不能拿改进版成本回答原流程。

23.7 7 · 图灵机递减

从最低位向高位找第一个 1，把经过的 0 改成 1，再把那个 1 改成 0；最后按题面要求复位读头。先测试 1、2、8 以及 0 的定义，0 若不允许递减必须报告边界。

23.8 8 · 允许原地不动

若一个可达 N 转移既不改符号也不改状态，就会永远重复同一配置，不能属于一个保证结束的有效计算路径。成本应按转移次数计，不只数移动次数；允许 N 并未改变可计算能力，模拟会带来至多常数层面的局部开销。

23.9 9 · RAM 与除乘

乘法可用移位加法，按位数计 O(log n)轮，字长模型须说明移位、加法是否单位成本。非负整数整除可用倍增减法，需处理除数 0；floor 与 ceiling 相差是否有余数的一项。RAM 寄存器/跳转抽象与 C++类型、对象生命期并不一一相同。

23.10 10 · 小 o 与小 ω

n=o(n log n)，n log n=ω(n)。看比值趋向 0 还是无穷，不用“大概差一级”判断；同阶函数不是彼此的小 o。

23.11 11 · 基本运算的成本

若模型明定机器字乘除为原语，可计 O(1)；若靠基础指令实现，或操作数是任意长整数，要按算法与位长计费。一个问题的答案可以因模型改变，前提必须先写。

23.12 12 · 多项式与对数

先对原题函数取值条件作约束，例如 f(n)≥1。若 log f(n)=O(log n)，则 f(n)≤n^C，是多项式上界；反过来也成立。若把 O 改成 Θ 或要求固定次数的单项式，结论需重新检查，不能只做形式替换。

23.13 13 · 指数实例

枚举 n 项所有子集有 2^n 个输出候选；枚举 n 个布尔变量所有赋值同样如此。区分“这个枚举算法指数级”与“问题已证明只能指数级”，后一结论强得多，不能乱下。

23.14 14 · 十进制位数

最大 w 位无符号数为 2^{w-1} floor(log10(2^w-1))+1。w=32 得 10 位，w=64 得 20 位。用对数估位数，不要先把巨大十进制数手写出来再数。

23.15 15 · 时间预算

记录平台、优化级别、输入规模、最坏数据和内存限额。每秒操作数只作粗估，排序比较、缓存失效和大整数运算成本不同；先排除明显不可能的阶，再做有代表性的基准。

23.16 16 · 迭代模板

把《习题解析》1-32 视作外部题目引用；本地没有该书全文，不能声称已逐个核对。正文 02 提供等差、几何、调和及对数级数；练习时把每种循环写成求和，分别估上/下界再对照原书。

23.17 17 · 分数与整数计数

用 floor(n/i)代表真实整数次数，用 n/i 夹逼即可；累计取整误差至多 n，不影响 n log n 主阶。分数是在估界，不是在声称执行了小数次指令。

23.18 18 · 调和级数

对递减函数 1/x，用矩形面积夹住积分，得 ln(n+1)≤H_n≤1+ln n。上下都是 log n 量级，所以 Θ(log n)，不必依赖记住欧拉常数。

23.19 19 · Stirling 与 log(n!)

Stirling 给 n!约为 sqrt(2πn)(n/e)^n，因此 log(n!)=n log n-Θ(n)。只求 Θ(n log n)时，后 n/2 项每项≥n/2，上界每项≤n，直接夹逼即可。

23.20 20 · 递归空间

线性 sum 深度 n，辅助空间 O(n)；二分 sum 同时只活一条深度 log n 的路径，O(log n)；普通迭代 sum 为 O(1)。若递归帧还复制子数组，空间要另加，不能只数层数。

23.21 21 · reverse 的先后

外端交换与内部区间递归访问互不重叠，先后交换不改最终结果，时间均 Θ(n)。但把递归放最后可形成尾递归形状；是否自动节省栈取决于语言与优化，可靠的 O(1)空间应显式写迭代。

23.22 22 · 主定理证明

展开递归树：第 i 层成本 a^i f(n/b^i)，共 log_b a)。把层成本和与叶成本相加；若层间几何衰减由根主导，几何增长由叶主导，相同量级累积 log n 层。正文 02 给分类，证明时不能丢掉叶层。

23.23 23 · 正则条件

由 a g(n/b)≤c g(n)、c<1，迭代反推 g(n)至少按 n^(log_b(a/c))，而非 log_b a)多一个正指数。反向不保证成立：可在多项式包络内构造起伏函数，虽有多项式下界，却在某些 n 上出现 a g(n/b)>g(n)。因此增长率与逐层规律不是同一条件。

23.24 24 · 带 log 因子递推

若 g(n)=Θ(n^k log^p n)，比较 a 与 b^k a>b^k 时叶主导；a<b^k log^p p>−1^k log^(p+1)n，p=-1 出现 log log n，p<−1 则不能沿用同一条公式。

23.25 25 · Akra–Bazzi

先列适用条件和方程 Σa_i b_i^p=1，而非 p 权重看作守恒分配，沿尺度汇总外部代价 g(u)/u^(p+1)得到积分项。完整有扰动版本需要处理舍入与正则性，不能只拿一次积分演算冒充一般定理的严密证明；本书演示的是无扰动、良性增长形式。

23.26 26 · 最大和区间的位置

维护当前区间起点、全局左右端；更新时用同一个并列比较规则。允许空区间与不允许空区间分别初始化。仅加常数个下标不改变线性版 O(n)时间、O(1)空间。

23.27 27 · 不等分最大区间

固定 α 与 1-α 均在(0,1)，每层总处理长度 O(n)，深度 O(log n)，故仍 Θ(n log n)。α 随 n 逼近 0 时不能照搬；固定每次分成 1 与 n-1 会退化。

23.28 28 · 记忆化

先找不同递归路径是否重算同一状态，以参数元组作键；缓存的是完整状态答案，不是随手记某个局部变量。若状态还依赖已选集合，就不能只按当前位置缓存。

23.29 29 · LCS 恢复

整表保存后从右下回溯；相等字符走左上并记录位置，非等时向较大邻格走，最后反转输出。并列规则影响恢复哪一个 LCS，不影响长度。只保存滚动两行无法直接恢复全部路径。

23.30 30 · 自顶向下与自底向上

两者使用同一状态图。记忆化能避开未访问状态，但有递归和字典成本；填表连续、无递归，但可能填无关项。状态稀疏、依赖复杂时结论会变化，不能一概说某种总更好。

23.31 31 · 代码阅读

逐接口写前置条件、后置条件、规模变化和所有权；测试空输入、单项、全等、逆序、边界索引。源码包的比较器、区间闭开和教材版本要对齐，不能只看函数名就认定一致。

23.32 32 · 演示验证

先预测下一步的栈、指针或数组状态，再播放一步核对；记录一个违背自己直觉的过程并解释。演示是帮助理解的实例，不替代对任意输入的正确性证明。

24 PS02：向量与查找，43 题逐项检查

打开教师原始习题单。下面按原题号逐项说明。符号有歧义时，以原题所用接口、区间与概率模型为准；测试/演示题给出可以执行的检查方法，不伪造已经完成的硬件实测。

24.1 1 · 面向对象基础

对象把状态与操作放一起；构造建立合法状态，析构回收资源；模板把元素类型参数化；函数对象是可调用的对象。重载按参数签名区分，重写涉及继承中的虚函数，二者不要混。先用一个保存两个整数的小类验证各概念。

24.2 2 · STL vector

实测 size、capacity、reserve、resize、push_back 与 erase 的区别。reserve 只增容量，不凭空产生可下标访问的新元素；resize 会改变 size。保存元素地址后扩容再使用旧地址，是需要专门检测的失效情形。

24.3 3 · 课程 Vector

对照构造、复制、析构、插入、删除、扩缩容的调用链，每一步检查 $0 \le size \le capacity$。测试报告里的随机通过不替代接口不变量；同时检查空区间、自复制和资源释放。

24.4 4 · 批量与单项

批量版本把后缀一次搬开，再写 k 项；反复单项插入会重复搬相同后缀。以长度 1000、在头部插 100 项手算移动量，两种接口可能相差近两个数量级。

24.5 5 · insert 成本

一次批量插入的主要工作为 k 项复制和 n-r 项移动，$\Theta(k+n-r)$，扩容时还要复制原数据。逐项插入在同一附近位置可能 $\Theta(k(n-r)+k^2)$，位置更新策略也影响公式，先明确调用顺序。

24.6 6 · 搬运方向

右移从后往前，左移从前往后，避免先覆盖尚未读的源。memcpy 要求不重叠，memmove 允许重叠；但任意 C++非平凡对象不能直接当裸字节搬，需要尊重构造、移动和析构语义。

24.7 7 · remove 成本

删除[lo,hi)时后缀长度 n-hi 决定搬移量；lo=hi 应直接返回，避免原地赋值。是否值得增加分支取决于空删除频率与移动成本，阶相同不表示常数无所谓。

24.8 8 · 扩容比例与批量

任意固定 $\alpha>1$ 的几何扩容给每元素摊还常数搬迁；α 越接近 1，复制常数约随 $1/(\alpha-1)$增长，空间浪费较小。批量插 k 项至少 $\Theta(k)$，按调用数平均与按元素数平均不能混用；STL 增长倍数须查实际实现，不是标准规定。

24.9 9 · 摊还对象

必须把同一合法序列中所有 expand 调用纳入，包括立即返回的便宜调用。只统计真正扩容的那些，就丢掉了两次昂贵搬迁之间积累的信用，已改变所求平均的含义。

24.10 10 · 缩容阈值

半满就缩容易在边界一插一删反复搬迁；四分之一才缩为一半，给下一次扩容留下大量操作间隔。构造“插到刚满、删一项、再插”的序列检查抖动。

24.11 11 · 去重与哨兵

当前项本就处在查找区间末端外的哨兵位置，可省重复保存/写回；每轮省常数不改最坏二次阶。删旧项或新项会改变保留第一份/最后一份的语义，也会改变扫描指针的更新。

24.12 12 · 减少移动

用读指针扫描、写指针构造无重复前缀：在已保留区 find 判断，未出现才写。移动至多 O(n)，比较最坏仍 $O(n^2)$，全相同时可线性。若允许排序或额外哈希/位图，可改进最坏或期望界，但保序、关键字模型和额外空间也随之改变。

24.13 13 · 函数对象

写 Sum 对象含累加状态与 operator()，遍历后读结果；注意接口是按值复制还是按引用传入，按值会让外部状态看起来没变化。空向量应返回初始累加值。

24.14 14 · 用遍历做冒泡

函数对象保存上一项位置和交换标志，对相邻项实施比较；一次 traverse 完成一趟。仍需最多 n-1 趟，最坏 $\Theta(n^2)$，并没有因为接口抽象而改变核心工作量。

24.15 15 · 判等与比较

排序比较器表达严格先后；等价常定义为!less(a,b)&&!less(b,a)。元素的 operator==可能比较全部字段，而排序只比较 key，两者未必一致，去重必须选与业务相符的等价关系。

24.16 16 · 缩容后访问

若仅保留秩，每次通过当前_elem 重新寻址，可随底层迁移继续工作；若缓存旧元素指针/引用，则可能悬空。把“下标还有效”和“旧地址还有效”分开检查。

24.17 17 · 惰性去重

写指针只有遇见新值时前进，读指针每轮前进；输出前缀始终是已读区的去重结果。可先跳过完全无重复前缀，避免自赋值，但增加判断也有成本，需实测而非保证更快。

24.18 18 · 本题的稳定约定

题面要求保留每组最后一个等价项。可从右向左压缩，再把结果置回，或顺向扫描每组到最后再输出。必须保留整条记录，不仅复制 key；这与排序稳定性“保持相等项相对顺序”是相关但不同的要求。

24.19 19 · 分支次序

六种比较次序最坏阶不变，但命中概率、两侧概率、比较成本、分支预测会影响常数。对 32 位值域中稀疏百万项，随机查询时“等于某项”的概率可能很低；先测相等未必划算。不能把随机查询分布当唯一实际分布。

24.20 20 · 比较模型的 1 比 2

课程按关键字比较次数计费，向左一比较、向右常需两比较；索引加减与循环判断被视作较低或统一开销，不是现实 CPU 完全免费。若把所有指令一律计数，常数比例当然会变，要先声明模型。

24.21 21 · 中点溢出

合法 $0 \le lo \le hi \le INT_MAX$ 时，lo+(hi-lo)/2 不超过 hi，可避免 lo+hi 溢出。即使当前内存放不下极大数组，库代码也不应依赖某台机器的内存偶然限制；还要检查 size 转 int 是否超范围。

24.22 22 · 重复项

版本 A 返回哪个命中由中点轨迹决定，不是运行时真正随机。用[2,2,2,2]手走不同区间长度即可看出差别；需要首/末命中应改成边界搜索。

24.23 23 · Fibonacci 切分

固定 λ 保证两侧按比例缩小。若左转成本 1、右转成本 2，最坏比较预算对应子问题容量约 $C^{-1/C}-2$，令二者和 1 得 C 为黄金比，较大比例约 0.618 分给便宜侧；镜像比例一般不再最优。实际收益还需扣掉计算轴点的成本。

24.24 24 · 版本 B 的特判

它若把最终唯一候选直接当作≤e 的前驱，就需要初始保证最小项不大于 e；A 做相等匹配、C 用统一边界不变量时可自然处理失败。不能脱离具体返回语义说所有二分都必须同一特判。

24.25 25 · 空区间

C 若返回最后一个≤e 的位置，可统一得到 lo-1 一类失败哨兵，但不可随后解引用。空区间初始 lo=hi 应零次循环；B 若读 S[lo]则必须先判空。

24.26 26 · 版本 C 正确性

除不变式证明外，可对区间长度归纳：长度 0 结论成立；每次比较排除一侧不可能答案，剩余区间严格更短，利用归纳假设。必须同时证明返回的边界满足左右两侧条件。

24.27 27 · 平均查找长度

先画所用 C 版决策树，区分成功关键字与失败间隙。n 个有序键有 n+1 个失败间隙，概率模型可选各键/各间隙等概率，不能混在一个平均里。精确常数由树形和 n 的二进制长度决定。

24.28 28 · 各版本与失败叶

验证长度更新 floor/ceil 后最后只能到 0 或 1；失败间隙总比键多一个。对 n=2^k-1 画满比较树，用各深度节点数加权求平均，别把最深路径当平均路径。

24.29 29 · 终止

以 hi-lo 为非负下降量，每种分支都要严格缩短并保持 lo≤hi。特别检查长度 1 时 mid 等于哪端；lo=mid 却不+1 是常见无限循环来源。

24.30 30 · 插值最坏例

把大量值压在一小段，末尾放一个极端大值，估算轴点会总靠近一端，一次只排除很少元素。分布假设不成立时应转用二分或设置回退机制。

24.31 31 · 非均匀分布

若已知连续分布 CDF 为 F，可把值 x 映射到累计概率 F(x)，再估算其秩；参数估计误差、离群点和离散重复值都会影响预测。模型只是猜轴点，正确性仍靠区间边界检查保证。

24.32 32 · 混合查找实测

固定机器与编译选项，分别生成命中/未命中、均匀/聚集数据，逐渐缩小查找表比较中位耗时。报告阈值和误差，不从别人的 CPU 借一个“通用阈值”。本稿提供实验方法，没有伪造具体纳秒结果。

24.33 33 · 冒泡提前终止

每趟先令 sorted=true，发生交换再设 false；已有序输入一趟结束。用逆序、已排、仅末尾有小项和全等数据追踪标志，平均节约量必须指定随机模型后实测。

24.34 34 · 冒泡跳跃

记录最后发生交换的位置，后方已无需再比；每趟缩小活动后缀。构造只有前面乱、后面已序的数据观察收益，仍保留逆序二次最坏情况。

24.35 35 · 非对半归并

固定 α∈(0,1)按 αn 与(1-α)n 切分，仍 Θ(n log n)。α 过分靠边会增加常数；若 α 依赖 n、导致每次只减 1，则可退化二次。

24.36 36 · 复用缓冲

排序开始时申请一份足够大的辅助数组，各层传区间使用，结束统一释放。省去反复 new/delete，不改变 O(n)辅助空间阶，注意不能让两个同时使用的区间相互覆盖。

24.37 37 · 另一边耗尽

若实现只复制左半，左半耗尽后，右半剩余项往往已在正确位置；如果两边都放入独立缓冲，则都要复制剩余。必须按真实源/目标重叠关系解释，而不只背一个 merge 模板。

24.38 38 · 整段已有序

先检查左段最大≤右段最小，成立可直接返回；需两段非空。多一次比较换来省整段扫描，已序数据收益大；对全随机数据是否更快，需测常数。

24.39 39 · 原地归并的代价

标准接口名 inplace 不等于实现绝不申请缓冲。旋转/递归等原地归并可减少额外元素存储，但可能增加比较或移动，也影响复杂度界。分清“某个库实现”的行为与标准保证，不能把题面的概述当全部版本公理。

24.40 40 · 位图复位

普通位图按字清零需与值域 U 相关；校验环用 dense/sparse 与逻辑 size 实现 O(1)清空。时间戳也是办法，但时间戳溢出需处理，且首次数组初始化仍计成本。std::bitset 的容量是模板参数，动态 Bitmap 另有扩容语义。

24.41 41 · 位图去重与排序

插入 n 项 O(n)，按值域扫描输出 O(U)，总 O(n+U)。当 U=O(n)可写 O(n)；若 U≫n，不能继续声称与 n 线性。还应检查负数的偏移映射与内存预算。

24.42 42 · 校验环删除

删 x 时把 dense 最后一个有效值 y 搬到 x 的位置，同时令 sparse[y]指向新位置，再 size−；若 x 本来就在最后，只减 size。旧 sparse[x]不必清零，因为成员检测还会验证反向一致性。

24.43 43 · 大值域的稀疏集合

只遍历 dense 中实际出现项，可使去重扫描不必清整个值域，但底层 sparse 仍可能占 O(U)空间，输出也不自动按键排序。省的是重复清零/扫描时间，不是凭空让所有成本与 U 无关。

25 PS03：列表与排序，37 题逐项检查

打开教师原始习题单。下面按原题号逐项说明。符号有歧义时，以原题所用接口、区间与概率模型为准；测试/演示题给出可以执行的检查方法，不伪造已经完成的硬件实测。

25.1 1 · 静态与动态

按逻辑集合、顺序、链接关系是否改变来定义，不只看物理地址。比如游标列表的节点留在原数组格，改 succ 下标也能改变逻辑顺序。

25.2 2 · 秩与位置

秩是序号，位置是节点句柄；列表 p+k 通常需要走 k 步，重载运算符只是简化写法，不会创造 O(1)随机访问。把便利语法误当低成本，是本题重点。

25.3 3 · 节点距离

若前后未知，可从两端分别沿合法方向交替探索，遇到对方即停止，已知双向且同表时成本与距离成正比。先确认同表和有效位置，否则可能走到哨兵仍找不到。

25.4 4 · 倒数第 k 项

双指针先拉开 k 步，再同步走，避免先完整统计再从头定位。两者同为 O(n)，但实际读取步数与缓存不同；k 非法时要报告，不能从空指针继续走。

25.5 5 · 哨兵

空表也保有统一前驱/后继边界，插删无需每次分别处理“没有左邻居/右邻居”。测试空表插首项、删唯一项、接入首尾与重复清空，哨兵本身不能当数据删掉。

25.6 6 · 链接修改顺序

先保存所有之后还会用到的旧邻居，再修改指针。画出每条赋值后的图，检查是否丢失剩余链、是否形成错误环；等价的重排必须证明读到的仍是原值。

25.7 7 · 节点移动到自身

this==s 时应按接口定义直接返回，或明确禁止该前置条件。若盲目先摘下再插到已失效邻居关系处，可能把链表接坏；相邻节点的移动也需专门测试。

25.8 8 · 复制顺序

从尾到头读旧表并 insertFirst 可保序；从头到尾读并 insertFirst 会反序。深复制仍 O(n)时间和 O(n)新节点存储，使用循环可避免递归栈。

25.9 9 · 单链析构

每次先保存 next，再 delete 当前，再前进；不必找前驱，仍 $\Theta(n)$。如果每次都从头找尾再删，就会无谓退化为 $\Theta(n^2)$。

25.10 10 · 无序去重

维护已处理的无重复前缀；发现重复时明确删旧还是删新，并避免删掉当前扫描节点后再访问它。全等列表每次短查并常数删除，可到 $\Theta(n)$最好；互异输入最坏 $\Theta(n^2)$。向量原版还要付后缀搬移，不能直接照抄结论。

25.11 11 · 遍历

把访问函数用于求和、打印、验证有序性与统计。若访问器会删除当前节点，普通 traverse 未必支持，要使用安全遍历协议并提前保存后继。

25.12 12 · 用遍历做 find/search

访问器记录是否命中、命中节点和比较状态。有序查找可提前终止，但通用 traverse 若不支持中断，仍会扫完；抽象接口的能力会影响常数和可表达行为。

25.13 13 · 有序去重

少于 2 项时没有可比较的相邻数据，特判可避免触碰哨兵。也可把循环条件写成“当前真实节点与其真实后继都存在”，使边界自然退出。全等、单项、空表都要测试。

25.14 14 · 失败前驱

search 返回头哨兵表示没有满足前驱条件的元素；插在该位置后仍有序。若要确认目标存在，必须排除哨兵并检查等价关系，不能把任何非空返回都当命中。

25.15 15 · 有序列表搜索界

在不增加索引且只靠相邻链接的模型中，最坏必须经过线性多个节点。减少关键字比较不等于减少指针移动；加跳转层或树索引后已是另一种数据结构。

25.16 16 · 交换与平移

交换 data 避免重新分配节点，常数成本；摘除再 new 会产生分配/析构成本且可能破坏外部句柄。若采用摘下同一节点再接入，则无需重新构造元素，语义又不同。

25.17 17 · getMax 与稳定性

从后向前仅在严格更大时更新，可选最后一个最大项，配合稳定的节点移到末尾可保持顺序。但当前课件选择排序主体若用任意 data 交换，仍不稳定：[(2,A),(1,B),(2,C),(1,D)]可让 D 跑到 B 之前。原页把“选最后最大”与整体稳定放得很近，阅读时必须把实际 swap 与 move 两种实现分开。

25.18 18 · 向量选择排序

每轮扫描选最大，比较 $\Theta(n^2)$，至多一次交换，故移动 O(n)。它适合元素移动昂贵而比较相对便宜的情形，但普通交换版本不稳定。

25.19 19 · 交换与轮换

把排列看成若干互不相交的置换环。交换同一环中的两个位置会把一个环拆成两个；交换不同环中的位置会把两环合一。用(1 2 3)与(4 5)的小图分别追踪验证。

25.20 20 · 交换法的期望轮换数

每次把一个未就位最大元素交换到最终位置，相当于从其环中剥离一个固定点。含 c 个环的排列最少 n-c 次有效交换；均匀随机互异排列中 E[c]=H_n，可用“第 k 项成为某环代表”的指示变量概率 1/k 求和。

25.21 21 · 平移法不逐输入等价

[3,1,2]有一个置换环；先把 3 平移到末尾后已全序，后面可能出现更多原地就位，因此不能逐排列照搬“原地次数=环数”。在均匀随机模型下，剩余相对顺序仍均匀，每轮最大值位于末端的概率 1/k，所以某些期望结论仍可相同，需另证。

25.22 22 · 在线与输入敏感

插入排序每来一张牌就能维护已读前缀有序，且近乎有序时移动少；选择排序通常要看完整待选集合，比较数不因已序变少。它的优势是控制总交换/移动次数。

25.23 23 · 向量插入成本

数组必须为插入位置腾空间，最坏移动 $\Theta(n^2)$；链表定位虽也可 $\Theta(n^2)$比较，却每项只需常数次改链接。二分查位置不能消除数组搬移。

25.24 24 · move 与删建

move 保留同一个节点及元素身份，只改链接；remove 后重新 insert 若销毁重建，会多分配并可能使外部引用失效。比较性能时必须先确认 remove 到底返回元素值还是只是摘链。

25.25 25 · 最好平均情况

互异均匀随机排列中完全有序概率 1/n!；平均逆序对 n(n-1)/4，所以通常二次时间。原地不动次数可按各前缀新元素恰为最大/最小的相应事件求期望，方向与相等处理决定具体事件。

25.26 26 · 列表为何不二分

访问中点需要走链接，失去向量的 O(1)秩访问。若坚持二分，需能高效按秩定位并更新，通常要增广索引；仅改比较逻辑不能获得同样时间界。

25.27 27 · 逆序对

最多 n(n-1)/2，逆序排列取到；互异随机排列每对以 1/2 概率逆序，期望 n(n-1)/4。插入排序相邻搬移与这些逆序有关，所以平均 $\Theta(n^2)$。

25.28 28 · 冒泡趟数

趟数受元素向左所需的最大位移等因素影响，不只由逆序总数决定。构造一个最小元素位于末端与一个最大元素位于开头比较：逆序数可以同阶，所需趟数却差很大。精确极值应按原题 n 和 k 约束构造，不凭总逆序数直接除 n。

25.29 29 · 逆序交换与最少交换

交换一对逆序的非相邻元素会消掉它们之间的若干逆序，数量依中间值分布变化。任意排列按环分解只需 $n-c \le n-1$ 次任意交换排序；冒泡只许相邻交换，最坏需 $\Theta(n^2)$，两种操作模型不同。

25.30 30 · 不等长归并

无论 m 与 n 是否相差悬殊，总比较不超过 m+n-1，输出/接链总成本 O(m+n)。若只返回拼接链而不逐项复制，部分尾段可一次接入，但渐近上界仍线性。

25.31 31 · 用节点转移归并

直接 move 原节点避免重新 new/delete 和元素复制，并保留节点身份。相等时从左段先取可稳定，移动后原扫描指针应指向保存好的下一项。

25.32 32 · 向量与列表归并

向量连续、缓存好，但常需缓冲；列表可原节点重链接，缓存和分配局部性可能较差。两者 $\Theta(n \log n)$不意味着实际耗时相同。

25.33 33 · 切分比例

任何固定两边比例都保持 $\Theta(n \log n)$；若每次只剥一项再归并，递推 $T(n)=T(n-1)+\Theta(n)$，为 $\Theta(n^2)$，行为接近不断把单项插入/合并进有序序列。别把固定 1% 误当随规模趋零。

25.34 34 · 逆序计数

归并时右项先出，就增加左段剩余项数，O(n log n)。值域受限时可用计数结构/树状数组；是否更快取决于关键字模型，比较模型下不能无条件突破相关信息下界。

25.35 35 · 游标列表

用数组下标代替节点地址，空闲链管理未用格子。检查分配后摘空闲头、释放后接回、失效下标与重复释放；逻辑顺序不等于物理下标顺序。

25.36 36 · Java 版本

interface 规定能力，implements 提供实现；对象通常通过引用操作，内存由垃圾收集管理，但逻辑节点摘链仍须正确。GC 不自动修复错误链接，也不保证所有被无意引用的节点能回收。

25.37 37 · Python list

内置 list 是动态数组，尾 pop 摊还常数，头 pop/头插要搬后缀。若一个 reverse 用切片反复复制，另一个双指针原地交换，前者会多分配与复制；按实际代码逐行计费，而不把同名 list 当链表。

26 PS04：栈队列与单调结构，35 题逐项检查

打开教师原始习题单。下面按原题号逐项说明。符号有歧义时，以原题所用接口、区间与概率模型为准；测试/演示题给出可以执行的检查方法，不伪造已经完成的硬件实测。

26.1 1 · 向量栈顶互换

尾端作 top 时尾插/尾删成本低；头端作 top 每次需搬后缀，连续 n 次可 $\Theta(n^2)$。逻辑 LIFO 没变，但底层实现成本变了。

26.2 2 · 列表栈顶互换

双向列表两端都可常数次改链接，单向列表尾删通常要找前驱。先看实例代码是否双向、是否有哨兵与尾指针，再回答性能是否对称。

26.3 3 · 粉笔对应什么

访问标记数组、节点 status、记忆化表等都能记录“来过这里”。调用栈仅记录当前未完成路径，不会自动记住所有曾完成的状态；图有环时这一点尤其关键。

26.4 4 · 每帧节省与规模

若帧大小 s、栈预算 B，最大深度约 B/s。二分递归深度 $\log_2 n$，帧减半可让允许深度翻倍，理想规模从 n 到 n^2；减至十分之一可到 n^10。这只是在忽略输入存储等约束的模型内，不是实际电脑忽然多出那么多内存。

26.5 5 · 一般去递归

显式栈保存参数、局部状态和返回阶段，可模拟普通递归调用。消去语言调用栈不等于消去所有辅助空间；通用模拟往往仍需与原深度相当的存储。

26.6 6 · 消去后一次递归

若第二次调用是真尾位置，可把它改成循环，第一支仍递归。它减少当前帧等待第二支的必要，但不把所有二分递归自动变成常数空间，仍要分析第一支同时活跃的深度。

26.7 7 · 阶乘累加器

把未完成乘法累积到 acc，反复 acc*=n、n−，最终返回 acc，显式循环 O(1)辅助空间。一般“递归后还有操作”能否这样压缩，要看操作可组合性、顺序与副作用，不是每种都可无损改成一个标量。

26.8 8 · 头递归的术语

关键看调用前是否产生了使参数/状态向基例推进的计算。convert(n/base)的实参计算本身已经改变子问题；若真正任何状态都不变便调用同一实例，则永远到不了递归基。不要把“递归调用写在函数前面”误判成必然溢出。

26.9 9 · 单链析构

递归到后继再删除当前，后序销毁能保留未处理链；但栈 O(n)。迭代先保存 next 再释放当前，同为 $\Theta(n)$ 时间、O(1)辅助空间，长链更稳妥。

26.10 10 · 自定义括号

先词法识别左/右记号，再用栈存类型和原位置；不匹配时报告当前右记号、期待类型与未闭合起点。不能简单把 JSON 字符串中的括号也压栈，转义、引号和注释必须先处理。

26.11 11 · 加法结合方向

优先关系表中“栈顶+与当前+”选择先归约则左结合，选择先移进则右结合。整数无溢出时结果相同，但浮点加法不保证结合律，改变计算顺序可能改变数值。

26.12 12 · 乘方方向

数学通常右结合：$2^{3}4$ 表示 $2^{(3}4)$。对应同类运算符相遇时继续移进，延后左边求值；若先归约就变成$(2^{3)}4$。两者语义不同，不能凭加法经验调整。

26.13 13 · 右括号的空白行

右括号通常被匹配消去而不作为等待运算符长期入栈，因此以其为栈顶的若干关系不应出现。空白格表示非法组合/不可能状态，不是“默认随便选一种优先级”。

26.14 14 · 双阶乘

间接方案先把!!分词映射成独立 token，加入元数 1 与优先级规则；直接方案还要区分相邻两个!是一个双阶乘还是连续后缀阶乘，必须靠语法约定，不可只靠字符逐个执行。

26.15 15 · 操作数顺序

二元运算先弹 b 再弹 a，计算 a op b；一元只弹一项。执行前检查栈元素数，除数 0、阶乘的定义域与数值溢出要报告；发现错误后继续压一个随便的值，会让后面错误更难定位。

26.16 16 · 栈不会被运算符种类数限制

重复括号与待处理右结合运算可任意嵌套，栈深可 $\Theta(n)$。相邻优先关系是表驱动局部关系，不能随意假设传递性后把栈压缩成固定少数类型。

26.17 17 · RPN 转换正确性

用表达式树看：每个子表达式都先输出两个子树，再输出运算符，即后序。运算符栈延迟输出，直到所需右操作数已经完整出现。手工法更像显式解析树，在线栈法按读入过程逐步归约。

26.18 18 · RPN 的价值

转换一次后可反复在线性、低开销流程求值，也适合虚拟机或中间表示。若只算一次，总复杂度仍线性，不必夸成降阶；收益在语法处理与执行分离、表示简单。

26.19 19 · 长表达式练习

逐 token 画两栏“运算符栈/输出”，不要在脑中跳步。先定乘方、阶乘、双阶乘的结合与元数，再逐项转换。第一式的 RPN 为 0 1 + 2 + 3 - 4 + 5 + 6 + 7 8 + * 9 *；其余同用正文方法，尤其别把!!误转成两个!。本书附的检查器可按统一约定解析并对照，而不虚构某个未给出的优先表。

26.20 20 · 前缀 PN

可从右往左扫描：数字入栈，遇运算符取所需操作数，按左/右语义组合。也可递归解析“运算符后跟若干子表达式”。固定元数使边界可判，时间 O(n)，辅助空间最坏 O(n)。

26.21 21 · 禁形

按输入 1..n、输出弹出顺序约定，存在 k、i、j 且 i<j<k，顺序为 k...i...j，就不可行：弹 k 时 j 已在 i 上方，i 不可能先于 j 弹出。原题的 j+1...i...j 是这一机制的特例；若教材把输出画成反向栈，要先转成实际弹出顺序再对应模式名。

26.22 22 · 换分类视角

按最后弹出的元素/第一对配对括号分割，左右部分各自是独立合法栈过程，得到 Catalan 卷积 ΣC_k C_(n-1-k)。换一个观察角度，必须证明分类互斥、穷尽且组合可逆，不能只写相同递推就当证明。

26.23 23 · 重复 S 的计数

按 C,Sa,T,D,Sb,A 依次输入、实际弹出顺序记输出。末项 Sa 要求 C 先出，剩下 4 项自由混洗，得 14 种；末项 Sb 要求末两项 A,Sb，也得 14 种。进一步以 C 开头、AS 结尾，两类各 5 种；把 Sa/Sb 都写成 S 后，两类交集 4 种，末项为 S 的不同字符串共 14+14-4=24 种。数字由合法栈过程递归枚举并用集合去重交叉核对，不能直接把有标号 Catalan 数除以 2。

26.24 24 · 混洗判定

按目标下一项，把输入依次压入直到栈顶匹配，匹配即弹出；若输入耗尽而栈顶仍不符，就无法继续。每项最多入出各一次，外 for 与内 while 总 O(n)，不是相乘 $O(n^2)$。

26.25 25 · 括号的一一对应

入栈写左括号、出栈写右括号，任何前缀左数≥右数且总数相等；反向读合法括号，遇左压下一个输入、遇右弹栈，唯一恢复操作过程。互异元素时输出决定唯一可行操作，因此对应成立；重复字符的可见输出不再一一对应。

26.26 26 · 枚举排名与等概率抽样

令 A(o,c)为已用 o 个左括号、c 个右括号后的合法完成数，允许 o<n 时加左、c<o 时加右；终点 A(n,n)=1。按字典序 DFS 枚举；取第 k 个时先看加左分支的完成数，决定进入或减去该数进入右分支；均匀抽样按两分支完成数比例选，才使每个完整串概率相同。大 Catalan 数需大整数。本方法解释任务原理，不冒称重现 Knuth 书中各算法的全部实现。

26.27 27 · 向量队列

循环数组加 head/tail/size 即可，不必每次头删搬数组；扩容时按逻辑顺序复制环上两段。容量为 0、绕回、恰好满和扩容后顺序都要测试。

26.28 28 · 队列应用

BFS、打印任务、事件缓冲都按先到先服务。若需要最高优先级先出，那是优先队列，不能因为名字里也有“队列”就替换为 FIFO。

26.29 29 · 三种摊还常数

聚合、记账、势能可选择不同放缩，所得 2、3、4 一类常数界并不相互矛盾。比较前先统一每个基本栈动作的成本和包含哪些操作；只有阶相同不能证明精确常数相同。

26.30 30 · 矩形与并列

蛮力逐柱向两边扫描，单调升/降等输入可到 $\Theta(n^2)$。把最靠左明确为左边界最小，再对同左边界定右边界规则；对每次候选都使用统一比较器，不能只以面积严格更大才更新就保证所有遍历顺序下的左优先。

26.31 31 · 一趟和两趟

一趟在右边界首次确定时就能结算，可流式推进并省一份完整边界数组；两趟先分别求左右边界便于展示与调试。二者都线性，但输出时机、额外数组与相等高度策略不同。

26.32 32 · 单趟线性证明

聚合：每柱入一次出至多一次。记账：插入时预存未来弹出费用。势能取当前栈大小，push 真实 1 加势能 1，pop 真实 1 减势能 1，摊还受常数控制。哨兵操作另计常数，不改变总阶。

26.33 33 · 峰值空间的期望

最坏递增高度令栈长 n，最好可常数；空间定义为整个过程 max_r S_r。随机互异高度下单步栈为记录值数量，E[S_r]=H_r，但不能直接交换 E 与 max。可用记录指标的指数矩加并合界证明 E[max S_r]=Θ(log n)；条件与过程见“概率与证明补充”，没有随机分布时只许保证最坏 O(n)。

26.34 34 · MaxQueue 时间

每项进入影子队列一次、从尾或头离开至多一次，所以总线性，摊还 O(1)，不需要随机输入。题名中的 Expected 不能让你误以为必须假设随机；这一时间结论更强。

26.35 35 · MaxQueue 空间

固定窗口 w、互异均匀随机顺序下，某一时刻影子队列期望大小 H_w。若求长为 n 的整个运行的峰值，不能自动仍写 O(log w)：足够长数据可能出现一整窗递减，峰值达到 w。先区分固定时刻与全程最大，再给概率模型和界。

27 PS05：二叉树与 Huffman，37 题逐项检查

打开教师原始习题单。下面按原题号逐项说明。符号有歧义时，以原题所用接口、区间与概率模型为准；测试/演示题给出可以执行的检查方法，不伪造已经完成的硬件实测。

27.1 1 · 路径长度

按边数计，根到自身长 0；连接两条首尾相接路径时长度直接相加，不用补减重复顶点。用节点数计也可以定义，但公式会多出偏移，必须全章统一。

27.2 2 · 深度与高度

节点 v 深度 d，从 v 到最深后代长 h(v)，拼接得到根到某叶的路径，故 d+h(v)≤整树高度。若 v 所在子树含整树一条最深路径则取等；不在最长路径上时可以严格小于。

27.3 3 · 三种表示

父指针便于上溯祖先、用深度对齐求 LCA；孩子表便于向下列举与删子树；两者都有更方便但要同步维护。删子树至少处理被删节点数量，不能因找根 O(1)就说整次删除 O(1)。

27.4 4 · 二叉树计数

按边计数 n1+2n2=n0+n1+n2-1，得 n0=n2+1；再把 n=n0+n1+n2 代入得 n=2n2+n1+1。真二叉树 n1=0，因此叶数为(n+1)/2；这与“满到同一层”的完美二叉树不是同义。

27.5 5 · 多叉树与二叉树

在本书左孩子/右兄弟编码下，一棵有序树的根没有右兄弟，因此任意根带右分支的二叉树一般对应森林，不一定对应单棵树。把根视作某虚拟父节点的某侧孩子，是统一接口的入边约定，不是说根有真实父亲；不要混成根的右指针含义。

27.6 6 · 有序森林对应

把每棵根按森林次序用兄弟指针串联，每棵内部再以首孩子/后继兄弟编码；反向先拆根的兄弟链，再递归恢复孩子列表。两种过程互为逆操作，建立一一对应。

27.7 7 · 补成真二叉树

每个单孩子节点缺失的另一侧补一个带标记的空叶，原叶保留，原双孩子不变。新单孩子数 0，新双孩子数 n1+n2，新叶数 n0+n1；代入 n0=n2+1 可验证新叶=新双孩子+1。保留标记才能唯一撤销补充。

27.8 8 · 高度更新提前停

只在一条修改路径上改变子树时，某祖先高度一旦没变，更高祖先看到的孩子高度也没变，可停止高度传播。插叶、删叶、接入/分离整棵子树都可按同一局部依赖论证，但维护 size 等其他信息不能跟着停止。

27.9 9 · 勤奋与懒惰

频繁查高度时缓存 height 便宜，修改需沿祖先维护；偶尔查 size 则临时遍历可省长期维护。若查询频繁而更新少，缓存 size 合适；反过来频繁改动而很少查询，懒计算可能更合适。以实际操作序列算总成本。

27.10 10 · 中序头尾递归

全右链中先访问根再递归右支，可变为尾递归；全左链中先递归左支再访问根，接近头递归。判断要看剩余工作，不只看源代码中递归调用所在行。

27.11 11 · 单链先序

退化树每节点最多一个孩子，访问后直接令 p=唯一孩子，循环到底，Θ(n)时间、O(1)空间。无需 parent，因为不需返回另一个尚未处理的分支；一般二叉树则没有这项简化。

27.12 12 · 用遍历求 size

访问器每到一个真实节点计数一次，任一种保证不重不漏的遍历都得到 n。其他只依赖单节点访问的任务也可复用；若需要子树结果汇总，则必须选合适次序或保存状态。

27.13 13 · clone 次序

后序可先复制左右子树再组装根；先序可先建根再接孩子；中序也可在访问左支后建根，但必须保存原树结构与已复制左结果。不能仅遍历输出值再按某次序随便插成 BST，那会改变原形状。

27.14 14 · 打印树

需要同时保存深度、左右分支身份以及哪些祖先层还有未结束的兄弟，才能画竖线和正确拐角。先拿一棵左右高度不同的小树对照，防止把节点缩进当真实边关系。

27.15 15 · 清空次序

后序先删孩子最直观；先序也可先保存孩子指针再删根，之后只沿保存的指针走。不能先 delete 根再读 root->left。正确性由对象生命期和剩余可达性决定，不是只允许一个遍历名。

27.16 16 · RPN 建表达式树

数字建叶入栈；二元运算弹右树、左树，建运算符根并压回；一元只弹一树。结束必须恰有一棵树。非法 token、操作数不够和最终剩多棵都意味着输入不是合法 RPN。

27.17 17 · 先序的对称版本

若沿右链深入且暂存左支，要维持根-左-右次序就不能直接把所有右链节点立刻输出；可调整待办任务栈的压入顺序。所谓“对称”也可能得到根-右-左，先检查要求是否仍是同一先序。

27.18 18 · 先序正确性归纳

按树规模：先访问根，压入的右子树按后进先出顺序恰好对应各左路径节点的后续任务，递归处理每棵更小子树。只按一条藤的长度归纳不够直接覆盖侧枝规模，除非归纳命题同时包含任意合法侧枝的处理保证。

27.19 19 · 空右子树

压栈前检查非空可消除无用入出栈；增加分支与减少空操作的净收益取决于实现和数据，阶仍 Θ(n)。不要把“少做某类操作”直接等同为所有机器一定更快。

27.20 20 · 先序空间

栈记录尚未访问的右子树根，数量受当前路径上留下右侧任务的左祖先数控制，至多 O(h)。单侧链可让实际待办栈仅常数而 h 线性；有右侧任务的长左链可逼近高度上界。期望比例须给树形/路径分布，不能无条件写 1/2。

27.21 21 · 中序归纳

“整棵规模变小”自然处理左右子树；若用藤长度，需把藤外挂的任意子树纳入命题，否则归纳步无法保证它们被正确访问。衡量参数必须真能覆盖所有递归子问题。

27.22 22 · 最后中序节点

最右节点无右子树；上溯时一直作为祖先的右侧后代，最终越过根得到空。检查 parent 访问前先判空，不能从根继续解引用不存在的父亲。

27.23 23 · 无 parent 的后序

用显式栈保存节点与“左右处理到哪一步”，或用 lastVisited 判断右支是否已经完成。模拟调用帧即可，仍 O(h)辅助空间；没有 parent 不代表无法后序遍历。

27.24 24 · 后序正确性

以栈帧状态表示：左未处理→左完成右未处理→两侧完成可访问根。每次状态只向前推进，弹出时两棵子树都已访问；按规模归纳保证不重不漏。

27.25 25 · 迭代遍历总时间

每节点入栈、出栈和有限次状态转换都只发生常数次，或每条边只有限次上下经过，所以 Θ(n)。某次寻找左藤可走 h 步，不能据此给每个节点都重复收 h 步费用。

27.26 26 · 层次遍历

入队节点是刚出队节点的孩子，深度只增加 1；FIFO 使同层先处理、下一层排后。每节点除根外恰由父发现一次，故入出各一次、总 Θ(n)。同层左右顺序依孩子入队顺序确定。

27.27 27 · 层次空间

队列长度受最大层宽 w 控制，处理跨层时最多包含相邻两层的一部分，仍 O(w)。近满的完全二叉树使 w=Θ(n)，单链仅常数。不能拿高度 log n 当 BFS 队列空间。

27.28 28 · 完全树的数组

零基孩子 2i+1、2i+2，父 floor((i-1)/2)。父子不总是物理紧邻，但层次次序与同层节点连续，通常比散落指针节点有更好的局部性；具体缓存命中仍依访问模式。

27.29 29 · 真树由先后序重建

互异标签下，先序首为根，下一项是左子树根；在后序中找到它，即得左子树范围，再递归左右。真树保证内部节点不会只有一支，消除左右歧义；哈希位置表可避免反复线性搜索，但要验证序列一致。

27.30 30 · 增强序列

先序含空标记可递归读根、左、右；后序反向读根、右、左同理。仅中序含空标记仍可能歧义：根 A 右子 B 与根 B 左子 A 都给 # A # B #。空标记不是给所有次序都加上唯一性的魔法。

27.31 31 · 遍历组合

互异标签、完整序列时先+中或后+中唯一；一般先+后不唯一；只有层序也不唯一。若标签重复，连先+中也需额外区分身份；每项“唯一”都带模型前提。

27.32 32 · 交换子树的成本

两棵不互为祖先、可合法交换的子树，总权 W1、W2，根深 d1、d2，交换后成本差为(W1-W2)(d2-d1)。若大权原来更深，交换可不增；祖先与后代不能直接作同样交换，否则结构可能不合法。

27.33 33 · Huffman 最优

先证明最小两权可放到最深兄弟位置而不增成本，再把它们缩为合权叶。若缩小问题存在更优树，展开会改进原树，与最优矛盾。按规模归纳得到每次取最小两项的贪心算法最优。

27.34 34 · 框架与容器

核心只依赖“取最小、插入合并项、判断剩余数”的优先队列接口；容器可从无序数组换成堆或双队列。接口隐藏具体布局，但每种实现的操作成本必须重新带入总分析。

27.35 35 · 栈队列的单调性

原始权重先排好，合并产生的新权重按生成次序不减，因此从两个有序来源头部取最小两项即可。若用栈保存原始权重，要让最小值处于栈顶；换队列时按升序放入。排序未预先提供时，总成本含 O(n log n) 排序，不能把合并阶段线性冒充无条件总线性。

27.36 36 · 平均排序下界

均匀 n!种互异排列对应比较树不同叶。对二叉决策树，叶深平均至少 $\log_2(n!)$，可由 Kraft 不等式与凸性证明；因此平均 Ω(n log n)。这需要输入分布或随机算法的量化方式，不能不说明“平均谁”。

27.37 37 · 最优编码构造下界

限定任意未排序正实权重、仅靠比较区分顺序的模型。取间隔很大的超递增权重并任意置换标签，最优树被迫逐层把最大权放最浅处，树的深度顺序能揭示除最小两项外的排序，至少有 n!/2 类可区分输出，比较树深 Ω(n log n)。已排序输入、受限整数值域或只求部分信息可绕过这一模型，故不能把下界扩大为所有表示和所有任务。

28 从算法到程序：把边界当成题目的一部分

28.1 先写一份可执行规格

输入范围决定整数宽度，输出要求决定是否需要恢复位置，时间限制决定哪些阶可行。明确空输入、重复元素、无解、多个最优解、负数和端点相等的语义。不要等写完代码才问“这里应该输出 0 还是-1”。课程的 PA 指引、白盒指引、模板与样例都要读，但它们不能替代每题正式题面。Lab1 目录还含个人调试代码、报告与提交材料；本书不把那些私人记录公开当标准答案。当前网络学堂资源枚举对作业总量仍未核实，独立 OJ 上的所有题目数量也未由本稿证明完整。

28.2 正确性、性能、健壮性分开测

先写小规模朴素解作为对照，不追求快；再将优化解在数千小输入上逐个比较。用随机种子固定数据以便复现，再加入手造反例：空、单项、全等、严格升/降、极大/极小值、刚好跨边界、所有候选都无解。在 C++中用受支持的 AddressSanitizer/UndefinedBehaviorSanitizer 定位越界和未定义行为；它们会增加开销，性能测试应另用正式优化构建。未触发检测不证明没有错误，但检测到的问题不能用“最后答案好像对”忽略。

28.3 一个适合写报告的证据链

例如单调栈：先列每项入/出至多一次的证明，再给最坏递增输入达到线性栈空间的图，最后测 n 翻倍时耗时趋势。随机平均数不能代替最坏界；画出漂亮直线也不能证明数学复杂度。报告应区分理论、实现、实测，写清编译器、参数、CPU、数据分布及测量是否含 I/O。

28.4 本书附带的可运行检查

运行 `python3 示例/算法检查.py`，用独立朴素对照核验二分边界、直方图矩形、插删距离带状 DP、KMP 重叠、中间相遇方案计数、Huffman 小规模最优合并，以及 PS04 的 21 式解析与 RPN。固定种子下共 8006 次检查通过。这不是你 PA 最终程序的认证，也不是对任意输入的形式化证明，而是验证本书例子与实现相互一致的一层证据。

28.5 不要为性能牺牲问题定义

把相等项去重可能删掉合法方案数；把负权删除可能违反“至少选一个”；把半开区间改成闭区间可能多算端点；把 64 位乘法临时结果当 64 位一定够，可能先溢出再比较。优化前把这些约定写在代码最前面，测试时逐项故意撞边界。

29 几个不能只背结论的证明

29.1 摊还的势能到底是什么

记第 i 次实际成本为 c_i，操作前后势能 Φ_{i-1}、Φ_i，定义摊还成本 $\hat{c}_i = c_i + \Phi_i - \Phi_{i-1}$。把 m 步相加，中间势能消去：$\sum c_i = \sum \hat{c}_i + \Phi_0 - \Phi_m$。若 Φ_0 为常数、Φ 始终非负，且每步摊还成本有常数上界，则总成本 O(m)。这个等式就是“先攒信用、以后支付”的精确账本。

双栈队列令 $\Phi=2|\text{in}|$，把一次 push/pop 各计 1：入队实际 1、势能+2，摊还 3；out 非空出队实际 1、势能不变，摊还 1；倒 k 项再出队实际 2k+1、势能-2k，摊还 1。所以任意合法操作序列总线性，不需要概率模型。

29.2 选择排序与置换环

一个长 ℓ 的置换环可用 ℓ-1 次任意交换拆成固定点；各环加起来 n-c。每次交换最多把环数增加 1，因此至少也要 n-c 次，达到最小。随机排列的环数期望 H_n 可由递增插入构造看：把第 k 个新标签插入已有环结构时，有 k 种等可能方式，其中恰一种让它自成新环，新增环指标期望 1/k。线性求和得 H_n。

29.3 单调栈单步空间与峰值空间

假设 n 个互异高度的相对次序均匀随机，处理前 r 项后，递增栈保存该前缀从右向左的记录最小值。长度为 j 的后缀中，最左项成为新最小值的概率 1/j，因此 $E[S_r] = H_r = \Theta(\log r)$。

峰值 $M=\max_r S_r$ 不能直接把 E 移进 max。记录指示变量有经典独立 Bernoulli(1/j)分解，指数矩满足

$$E[e^{S_r}] = \prod_{j=1}^{r}\left(1 + \frac{e-1}{j}\right) \le \exp((e-1)H_r) \le Cr^{e-1}.$$

Markov 不等式给 $P(S_r \ge t) \le Cr^{\{e-1\}}e^{-t}$；再对 r=1..n 用并合界，$P(M \ge t) \le Cn^e e^{-t}$。令 $t=A \ln n+s$，取 $A>e$，尾部随 s 指数衰减；按尾概率求期望得 $E[M]=O(\log n)$。另一方面 $M \ge S_n$，所以 $E[M] \ge H_n = \Omega(\log n)$，合起来 $\Theta(\log n)$。这才是对峰值的完整论证，而不是交换两个算子。

若高度有重复、相关性或指定分布，上面的随机排列模型不自动成立；最坏仍可 n 项全在栈里。

29.4 最大队列的另一个坑

固定时刻、窗口 w 内高度为独立连续同分布时，影子队列是后缀最大记录，期望 H_w。可是观察无限长输入，某一窗恰好递减的概率 1/w!>0；用互不重叠窗口，出现至少一窗递减的概率随窗口数趋近 1，峰值趋近 w。因此“单窗平均 O(log w)”不等于“任意长运行的峰值平均也 O(log w)”。

29.5 平均比较排序下界

设 n!种等概率排列对应比较树叶，叶深 d_i。二叉树 Kraft 界 $\sum 2^{-d_i} \le 1$；凸性给 $2^{-\bar{d}} \le (1/n!)\sum 2^{-d_i} \le 1/n!$，于是平均深度 $\bar{d} \ge \log_2(n!) = \Omega(n \log n)$。若输入不是均匀分布，平均下界应与其信息量关联，不能继续机械用 log(n!)。

29.6 归纳证明的命题要足够强

证明树遍历时，只归纳“左藤有多长”可能没有同时约束挂在藤上的任意子树。修补办法是增强命题：对任意合法侧子树，算法都按所需次序完成各待办任务；或直接对整树规模归纳。归纳的困难常不在代数，而在命题没有包含下一步真正需要的事实。

30 综合练习：先自己走一遍，再对答案

30.1 1. 同一段循环为何不是二次

外层 i=1,2,4,...直到不超过 n，每层循环 i 次。答：总量 1+2+4+...<2n，因此 Θ(n)；迭代次数虽 Θ(log n)，每次代价不同，不能乘以统一 n。

30.2 2. 删除重复项的语义

记录[(1,A),(1,B),(2,C)]按 key 去重，要保留最后一份。答：结果应[(1,B),(2,C)]；若用普通顺向“遇新值就写”会保留 A，违反要求。可以按组扫描到最后一项再输出，仍 O(n)。

30.3 3. 二分遇到全等数据

a=[5,5,5,5]，lower_bound(5)=0，upper_bound(5)=4，区间[0,4)包含全部等值项。找最后≤5 的位置为 3，找第一个>5 的位置为 4；这些不同返回值都合理，前提是接口定义不同。

30.4 4. 链表插入很快，为什么排序仍慢

每次在已知位置改两条边可 O(1)，但在有序前缀寻找位置最坏 O(n)，总 n 次仍 $\Theta(n^2)$。数组二分定位虽快，搬移后缀仍可能二次。寻找和修改必须各自计费。

30.5 5. 直方图的相等高度

[2,2,2]最大面积 6。严格递增栈可以在相等时合并代表，也可保留全部相等项并按一致规则弹出；不能一边用“左侧严格小于”、另一边也任意写“右侧严格小于”却忘记并列柱的归属。用全等数组最容易暴露这一问题。

30.6 6. 树重建反例

先序[A,B]、后序[B,A]：根 A 左子 B 和根 A 右子 B 都满足，故普通二叉树不唯一。加中序[B,A]可选左子；中序[A,B]可选右子。缺少结构约束时，不能把“重建函数写出来了”当唯一性证明。

30.7 7. AVL 的 LR 情况

插 30、10、20。失衡根 30，重子 10，新增 20 位于 10 右侧。先左旋 10，再右旋 30，得到 20 根、10 左、30 右。每次画中间树并核对中序仍 10,20,30；只背“先左后右”却转错节点，会得到另一棵错误树。

30.8 8. 散列墓碑

m=7，10、17、24 线性探测放 3、4、5。删 17 把 4 直接设从未使用，查 24 时在 4 停下会误报；用墓碑继续探测即可。墓碑不是永久不清理，太多会拖慢查找。

30.9 9. Dijkstra 的负边反例

A→B=2、A→C=5、C→B=-4。若先确定 B=2，之后经 C 其实可得 B=1，破坏已确定距离不再改变的性质。图无负环也不救 Dijkstra 的非负前提；需要选适合负边的算法。

30.10 10. MST 不等于最短路径树

三点 A,B,C，边 AB=2、AC=2、BC=1。从 A 的最短路径树可选 AB 与 AC，总 4；MST 选 BC 再选任一条 2 边，总 3。两种优化目标不同。

30.11 11. KMP 的重叠

T=ABABABA，P=ABA，匹配起点 0、2、4。π=[0,0,1]，每次命中后 j 回到 1，让末尾 A 作为下一次前缀，不把已知信息清空。

30.12 12. 选第 k 小与多数

序列[1,1,2,2,3]中位数 2，但它只出现 2 次，未超过 5/2，不是多数。Boyer–Moore 投票所得候选必须在未保证多数存在时再计数验证。目标不同，复用算法前先核对输出定义。

30.13 怎样确认自己真的懂

对每题再改一个条件：允许重复、要求保序、允许负数、只给单链、图不连通、边权为 0、输入为空。若结论变了，说出是哪一步证明用到了旧条件。能指出这一句，通常比背出算法名字更有帮助。

31 PS04 的 21 个表达式：RPN 与检查过程

统一约定：乘方右结合，阶乘/双阶乘是高优先级后缀运算；!!表示双阶乘，除法采用精确有理数语义，非C++整数截断。老师具体程序若用别的优先表或浮点类型，须按其定义对照。下面 RPN 由可读解析树生成，再由独立栈求值与树求值交叉核对。

31.1 第 1 式

原式：`(0+1+2−3+4+5+6)∗(7+8)∗9`

```
0 1 + 2 + 3 − 4 + 5 + 6 + 7 8 + ∗ 9 ∗
```

依上述语义的值：2025。

31.2 第 2 式

原式：`(0∗1+2)∗3!/4!∗5∗6∗(7+8)∗9`

```
0 1 ∗ 2 + 3 ! ∗ 4 ! / 5 ∗ 6 ∗ 7 8 + ∗ 9 ∗
```

依上述语义的值：2025。

31.3 第 3 式

原式：`0!+(1+2)!!∗3−4−5!−6−7−8+9`

```
0 ! 1 2 + !! 3 ∗ + 4 − 5 ! − 6 − 7 − 8 − 9 +
```

依上述语义的值：-126。

31.4 第 4 式

原式：`(0+1+2)/3!/4!∗5!∗6∗(7+8)∗9`

```
0 1 + 2 + 3 ! / 4 ! / 5 ! ∗ 6 ∗ 7 8 + ∗ 9 ∗
```

依上述语义的值：2025。

31.5 第 5 式

原式：`(0!^1/2∗3!!/(4+5)∗6−7−8)∗9`

```
0 ! 1 ^ 2 / 3 !! ∗ 4 5 + / 6 ∗ 7 − 8 − 9 ∗
```

依上述语义的值：-126。

31.6 第 6 式

原式：`((0∗(1−2)+3+4)∗5∗6+(7+8))∗9`

```
0 1 2 − ∗ 3 + 4 + 5 ∗ 6 ∗ 7 8 + + 9 ∗
```

依上述语义的值：2025。

31.7 第 7 式

原式：`(0!−1!+2)∗3!/4!∗5∗6∗(7+8)∗9`

```
0 ! 1 ! − 2 + 3 ! ∗ 4 ! / 5 ∗ 6 ∗ 7 8 + ∗ 9 ∗
```

依上述语义的值：2025。

31.8 第 8 式

原式：`((0-1)^2*(3!!/4-5+6*7+8))*9`

```
0 1 - 2 ^ 3 !! 4 / 5 - 6 7 * + 8 + * 9 *
```

依上述语义的值：1647/4。

31.9 第 9 式

原式：`((0-1)^2*(3!!/4-5-6+7*8))*9`

```
0 1 - 2 ^ 3 !! 4 / 5 - 6 - 7 8 * + * 9 *
```

依上述语义的值：1647/4。

31.10 第 10 式

原式：`(0!+1+2)!+(3!-4)^(5+6)-7*8+9`

```
0 ! 1 + 2 + ! 3 ! 4 - 5 6 + ^ + 7 8 * - 9 +
```

依上述语义的值：2025。

31.11 第 11 式

原式：`(0+(1+2))^3/(4+5)*6!-(7+8)*9`

```
0 1 2 + + 3 ^ 4 5 + / 6 ! * 7 8 + 9 * -
```

依上述语义的值：2025。

31.12 第 12 式

原式：`((0-1)^2!*(3!!/4-5+6*7+8))*9`

```
0 1 - 2 ! ^ 3 !! 4 / 5 - 6 7 * + 8 + * 9 *
```

依上述语义的值：1647/4。

31.13 第 13 式

原式：`((0-1)^2!*(3!!/4-5-6+7*8))*9`

```
0 1 - 2 ! ^ 3 !! 4 / 5 - 6 - 7 8 * + * 9 *
```

依上述语义的值：1647/4。

31.14 第 14 式

原式：`(0!+1)*(2+3)*(4+5)/6*(7+8)*9`

```
0 ! 1 + 2 3 + * 4 5 + * 6 / 7 8 + * 9 *
```

依上述语义的值：2025。

31.15 第 15 式

原式：`(0!^1/2*3!!/(4+5)*6-(7+8))*9`

```
0 ! 1 ^ 2 / 3 !! * 4 5 + / 6 * 7 8 + - 9 *
```

依上述语义的值：-126。

31.16 第 16 式

原式：`((0-1)^2*(3!!/4-5*(6-7-8)))*9`

```
0 1 - 2 ^ 3 !! 4 / 5 6 7 - 8 - * - * 9 *
```

依上述语义的值：1647/4。

31.17 第 17 式

原式：`((0-1)^2!*(3!!/4-5*(6-7-8)))*9`

```
0 1 - 2 ! ^ 3 !! 4 / 5 6 7 - 8 - * - * 9 *
```

依上述语义的值：1647/4。

31.18 第 18 式

原式：`((0!+1!)^2+(3!-4)+5*6)*(7*8)+9`

```
0 ! 1 ! + 2 ^ 3 ! 4 - + 5 6 * + 7 8 * * 9 +
```

依上述语义的值：2025。

31.19 第 19 式

原式：`((0!+1)*2)!+(3!-4)^(5+6)-7*8+9`

```
0 ! 1 + 2 * ! 3 ! 4 - 5 6 + ^ + 7 8 * - 9 +
```

依上述语义的值：2025。

31.20 第 20 式

原式：`((0!^1+2)!!/(3!-4-5+6)-(7+8))*9`

```
0 ! 1 ^ 2 + !! 3 ! 4 - 5 - 6 + / 7 8 + - 9 *
```

依上述语义的值：-126。

31.21 第 21 式

原式：`((0!^1+2)!!/(3!-(4+(5-6)))-(7+8))*9`

```
0 ! 1 ^ 2 + !! 3 ! 4 5 6 - + - / 7 8 + - 9 *
```

依上述语义的值：-126。

32 从样卷提炼通用方法：不靠记住原题数字

本地样卷压缩包已拆出两份 PDF，结合原页核对了表达式、树遍历、摊还与归约等题型。其中资料带有课内使用标记，本书不转载其题面、原页或完整答案；下面用重新设定的例子讲通用方法。资料是否能在公开网站展示，与能否在本地读来学习，是两件不同的事。

32.1 1. 循环步长自己增长，用累计进度反推轮数

考虑 `i=0,j=0; while(i<n){i+=j; ++j;}`。执行 k 轮后，j=k、i=0+1+...+(k-1)=k(k-1)/2。退出需要 i 达到 n，因此 k 约√(2n)，时间 Θ(√n)。第一轮 i 不动不表示死循环，因为 j 改变，下一轮会有进度。实际 C++还须保证 i+j 不会在临界 n 附近先溢出。

32.2 2. 非几何扩容，把“容量轴”拿来积分

若容量 C 每次增加约√C，扩容一次搬 C 项，扩容次数在容量小段 dC 里约 dC/√C，所以累计搬迁量约 $\int_1^n C/\sqrt{C}\,dC = \Theta(n^{3/2})$。可用分段夹逼把估算变成严格界。更一般地，每次增 Θ(C^α) 0≤α<1 ¯ Θ(n(2-α))；α=1 的固定比例扩容回到线性总量。

这比只记“扩容有时贵、有时便宜”更具体：到底怎么增长，决定总成本。

32.3 3. 表达式树的栈峰值有递推

按左子树、右子树、根的顺序产生 RPN。叶子 S=1；二元节点有$S(T) = \max(S(L), 1 + S(R))$，因为算右边时左边结果还占一格。总节点数相同的树，峰值可能不同，不能只按树高或叶数直接猜。

若每个节点都满足左子树不小于右子树，则每次让栈多占一格的向右进入，会至少把剩余子问题规模减半，峰值受对数界控制。以内部节点数 k 计，达到栈峰值 s 至少需$2^{s-1} - 1$个内部节点：右边要达到 s-1，左边又不得比右边小。给 15 个内部节点，完美二叉表达式树可达到 5 格；向左退化的长树却只需很小的栈。

32.4 4. 逆序对有多少必须留在指定区间

长度 n 的序列，指定区间长度 m。全部可能的元素对有 n(n-1)/2，完全在区间内的对有 m(m-1)/2。因此区间外或跨边界最多可容纳两者之差的逆序对。若总逆序数 I 超过这一上限，区间内部至少有$I - [n(n-1) - m(m-1)]/2$对；若为负，下界截成 0。

这给的是可靠计数下界。要声称该下界可达到，还需构造同时满足总逆序数的实际排列，不能把“上界减法”自动当存在性证明。

32.5 5. 线性归约：映射要双向证明

把互异实数 x 映射为平面点(x,x³)。任取互异 a、b、c，AB 斜率$a^2 + ab + b^2$，AC 斜率$a^2 + ac + c^2$；两斜率相等当且仅当$(b - c)(a + b + c) = 0$。因 b≠c，所以三点共线当且仅当 a+b+c=0。

因此生成 n 个点只需 n 次固定形式映射，就把三数零和判定转成三点共线判定。必须同时证明：映射不会合并不同输入、真解能产生共线、共线不会带来假解、构造成本在所用实数 RAM 模型下线性。只证明“有零和就共线”还不够建立等价归约。

32.6 6. 遍历信息不完整时，先问能否唯一确定

有的题给 parent 和叶序，却不给 child 指针；有的给后序和层序。不要一开始就套普通递归模板。先定义每个时刻你知道的边、尚未返回的祖先和下一片待处理叶子；每次访问都要能从这些信息推出合法下一步。若想宣称线性，逐边记账：是否每条边只被走常数次？若某个祖先每遇一片叶子都从头重扫，就可能把高度额外乘进去。

本书不把某一种特定遍历对的唯一性泛化到所有树模型。二叉/多叉、有序/无序、标签互异与否、是否有空标记，都会改变可恢复的信息。

33 资料覆盖、原图出处与使用说明

这是一份独立组织、重新讲解的教材式初稿。课程范围首先按教师资料；并非教师审核的官方教材，也不把补充证明和预习部分自动列作考试要求。

33.1 章节覆盖表

教学来源	本书对应
教材/整套讲义第 1 章算法概论	01—03、22、28、31
第 2 章向量	04—06、23
第 3 章列表	07、24
第 4 章栈与队列	08—09、25、30
第 5 章二叉树	10—11、26、29
第 6 章 BST 与 AVL	12
第 7 章搜索树应用	14
第 8 章高级搜索树	13
第 9 章词典/散列	15
第 10 章图基础	16
第 11 章图进阶	16—17
第 12 章优先级队列	18
第 13 章串	19
第 14 章排序	20
已发布 PS01—PS05	22—26 逐题导航，题数 32+43+37+35+37=184
PA1 讲解 PDF 与 PPTX	21：Filename、Interview、Gift、Graphics、二维偏序
编程/白盒指引与模板	27：规格、复杂度、边界与验证
两份本地样卷	31：只提炼通用方法，不转载课内受限原题

33.2 原生解析与视觉核对

PDF 提取原生文字与页码，PPT/PPTX 提取文字和内嵌图片，同时原生兼容导出完整幻灯片以保留箭头、布局、曲线与公式。机器已为可用课件生成完整页级联系表；写入笔记的关键原图逐幅视觉核对。未声称已对数千页的每一个细节完成逐字人工审校。PPT 导出中缺失中文字体替换为 Noto Sans CJK SC，原件不覆盖。

原生提取和视觉复核需要互相补足：抽出的字不能告诉你箭头指谁，图片也不总能准确给出公式符号。课件中的省略或不一致处，本书在相关段落说明，不把它们无条件当作定理。

33.3 原图逐项出处

- `老师原图-向量.png`：`数据结构/slides/分章课件/02 向量.pdf`，PDF 第 29 页。直接渲染原始 PDF 页，不改图中结构。
- `老师原图-单调栈.png`：`数据结构/slides/分章课件/04 栈与列表.pdf`，PDF 第 95 页。直接渲染原始 PDF 页，不改图中结构。
- `老师原图-Huffman.png`：`数据结构/slides/分章课件/05 二叉树.pdf`，PDF 第 105 页。直接渲染原始 PDF 页，不改图中结构。

- 老师原图-增强遍历.png：数据结构/slides/分章课件/05 二叉树.pdf，PDF 第 87 页。直接渲染原始 PDF 页，不改图中结构。
- 老师原图-AVL.png：数据结构/slides/整套讲义/电子讲义(按章节).7z，DSA-2026F-Chapter/06.二叉搜索树.pdf，PDF 第 30 页。直接渲染原始 PDF 页，不改图中结构。
- 老师原图-KMP.png：数据结构/slides/整套讲义/电子讲义（按章节）.7z，DSA-2026F-Chapter/13.串.pdf，PDF 第 28 页。直接渲染原始 PDF 页，不改图中结构。

原创 SVG 图由工具脚本/教材/配图.py 生成，文字转为轮廓，可离线显示；它们是本书的补充示意，不伪装成教师原图。

33.4 版本和复核边界

2026-10-25，用新版 OneTHU 默认共享后台只读核验课程文件清单，三门课文件枚举均返回 complete=true。数据结构的综合资源接口对作业全量枚举仍给出未核实状态，因此本书不宣称覆盖了独立 OJ 未来/全部题目。未取得的外部《习题解析》全文也没有假装读过。参考编号保留为继续阅读入口。完整文字、公式与图在 Markdown 和 Typst 间共享；每次改正文后应运行构建器，Pandoc 公式转换警告将阻止成功构建。重要例题按推导、量纲、边界和独立小规模程序交叉检查，发现错误应同时修正文并重建两版本。

延伸阅读：MIT 算法课讲义。本书的章节编号与课程本地教材对应，不照搬另一门课的组织。